



计算机图形学与虚拟现实

冼楚华

Email: chhxian@scut.edu.cn

华南理工大学计算机科学与工程学院



课程信息

- 授课老师姓名：冼楚华
- Email: chhxian@scut.edu.cn 
- 个人主页: <https://chuhuaxian.github.io/>
- QQ: 89071086 （比较少用，非急事请不要私聊）
- 办公室: B3-202-2
- 课程QQ群（见二维码）



群名称: 2025计算机图形学@SCUT
群 号: 134660465

● 线段扫描转换

(Scan conversion of line segments)

- DDA算法 (DDA algorithm)
- Bresenham 算法 (Version 7–Sect. 8.9)

● 多边形扫描转换

(Scan conversion of polygon)

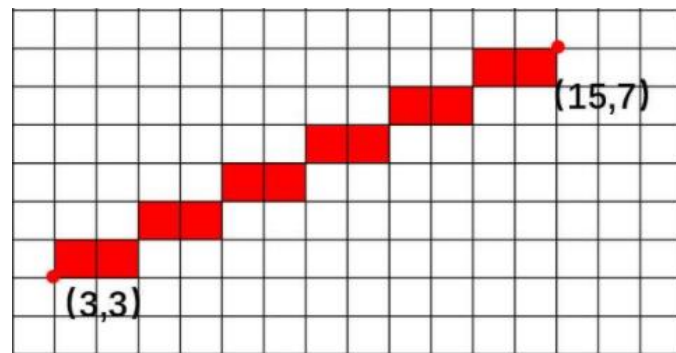
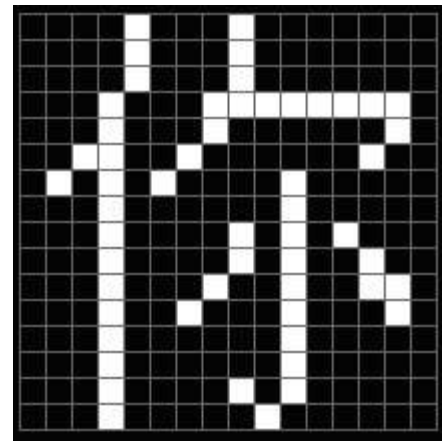
- 区域填充
- 多边形扫描转换

(Version 7: Sections 8.10)

- 线段扫描转换
 - DDA algorithm
 - Bresenham algorithm (Section 8.9)
- 多边形扫描转换
 - Fill-area algorithms
 - Scan-line polygon-fill algorithm
(Version 7 Section 8.10)

扫描转换(Scan conversion)?

- 几何图元的扫描转换
(线段, 圆, 多边形, 椭圆等)
 - 图形学所处理的图形都是用数学工具来表示, 在点阵设备上显示
 - 在点阵中找到基于数学表示的几何图元的最优近似点集
 - 本质上是一个采样问题

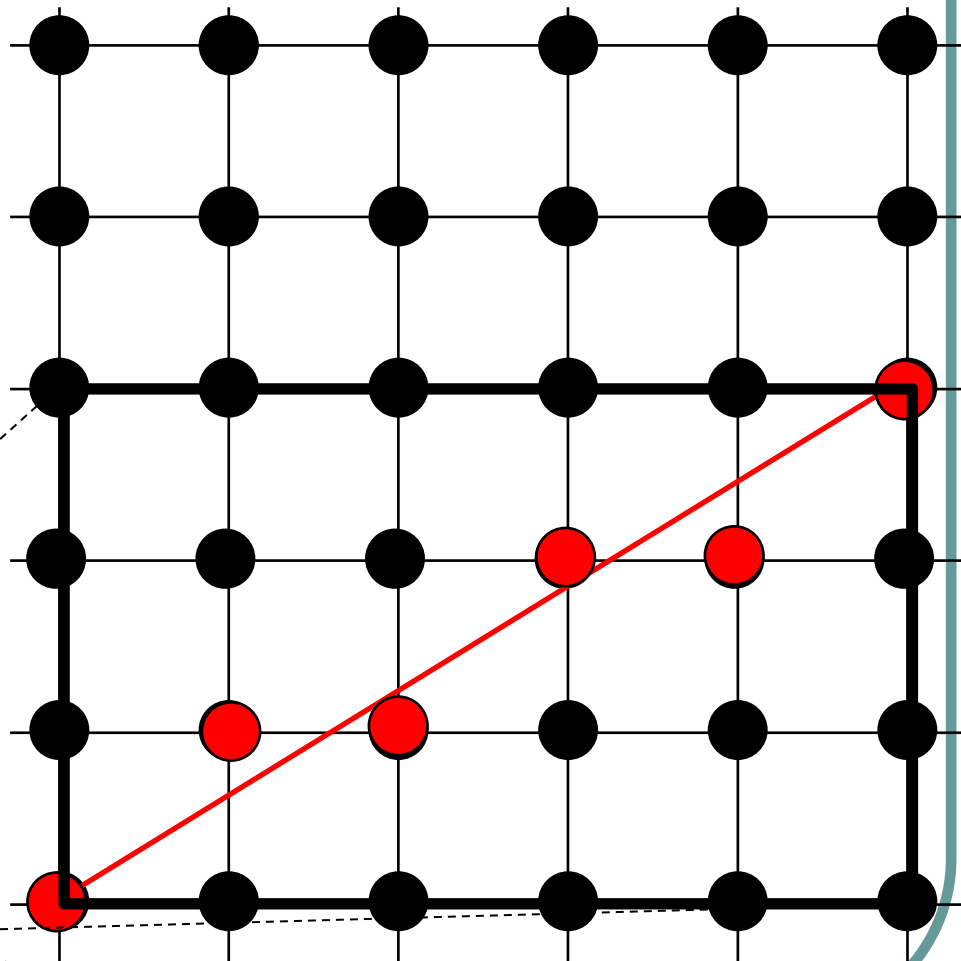
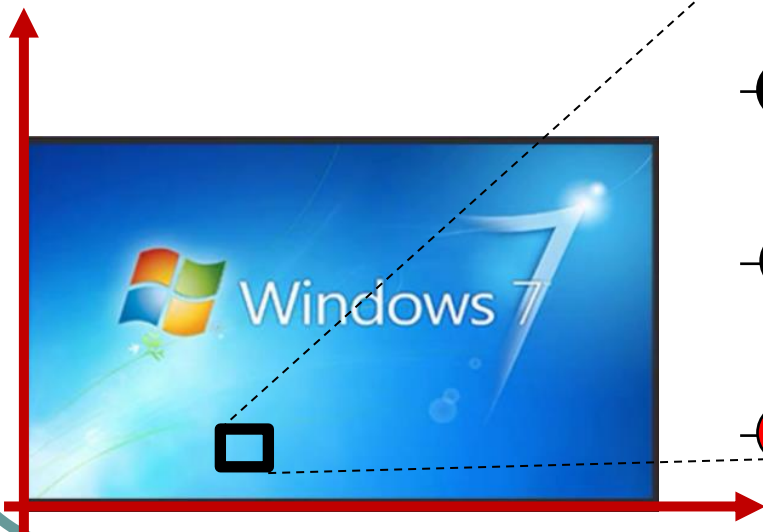


(Scan conversion A. K. A. Rasterization)

例: 线段 $(0, 0) — (5, 3)$

● 最优点集

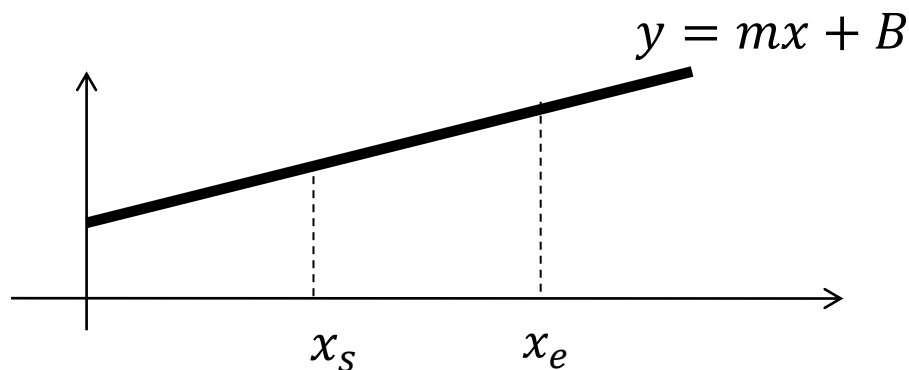
- 点到直线上距离最小
 - 某一行(列)上的点中到直线距离最小的点



直线方程

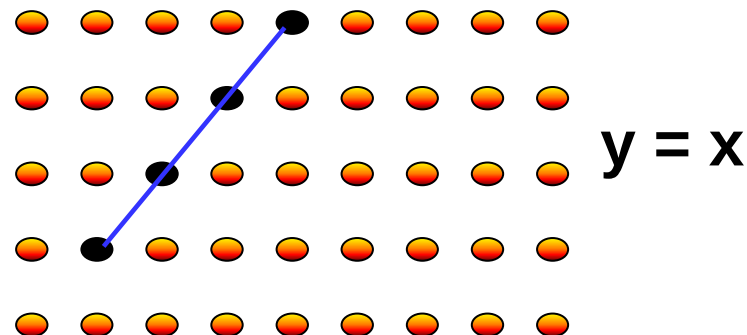
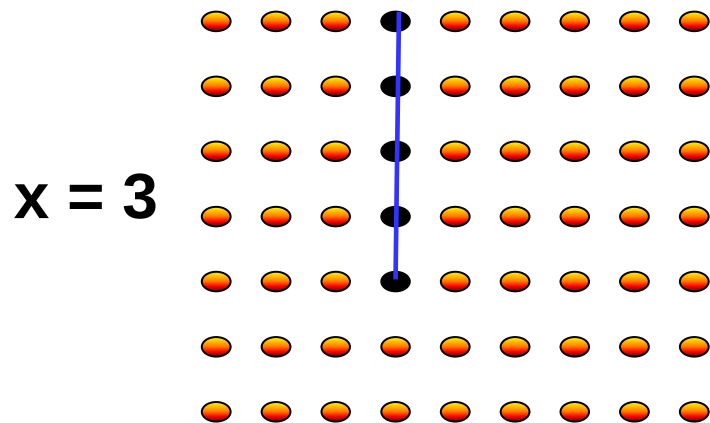
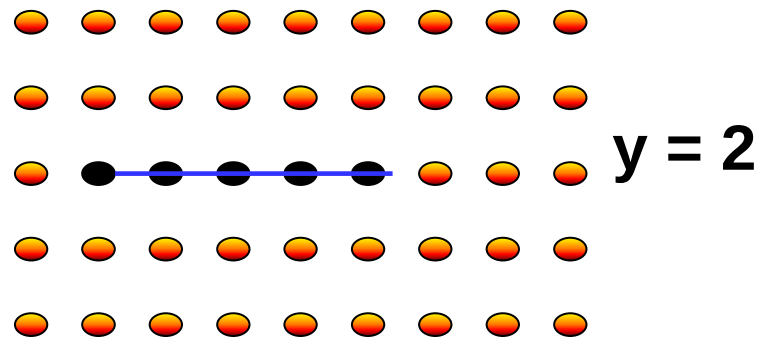
-- $y = mx + B, x_s \leq x \leq x_e, |m| \leq 1$

-- $x_s = a, x_e = a + n$ 均为整数,
 m 是直线的斜率



特殊情形：水平，竖直，45度

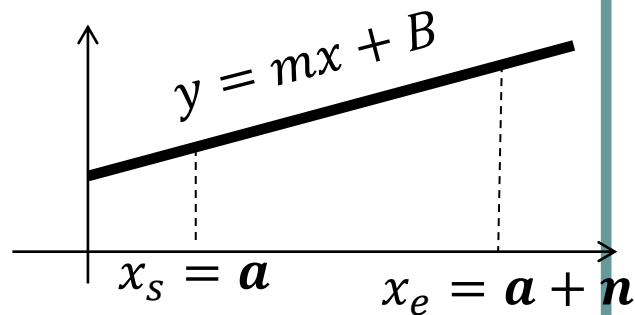
- $y = 2, 1 \leq x \leq 5$
- $x = 3, 2 \leq y \leq 6$
- $y = x, 1 \leq x \leq 4$



1 一个平凡的光栅化方法

- x 依次取如下的像素坐标 x_i
 $a, a + 1, a + 2, \dots, a + n$

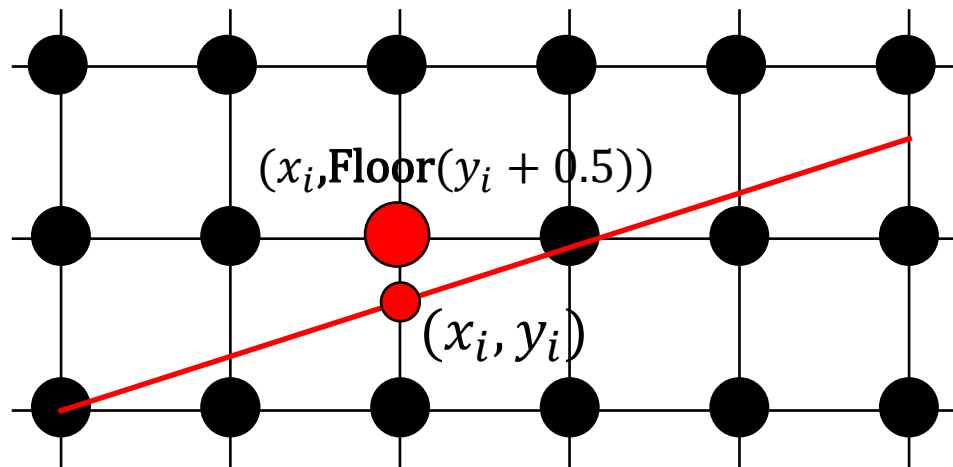
- 用直线方程计算相应的 y_i



$$x_i = a + i \rightarrow y_i = mx_i + B.$$

- 用像素点 $(x_i, \text{round}(y_i))$ 逼近上述直线段上的点 (x_i, y_i)

where $\text{round}(y_i)$ is the integer nearest to y_i



- 可以用取整数函数来计算最近的整数值 $\text{Floor}(y_i + 0.5)$

例子：平凡算法画线段(1,1) – (6,4)

- 线段方程

$$y = 0.6x + 0.4, x = 1, 2, 3, 4, 5, 6$$

$$x_1 = 1, y_1 = 0.6 \times 1 + 0.4 = 1, \text{ draw } (1,1)$$

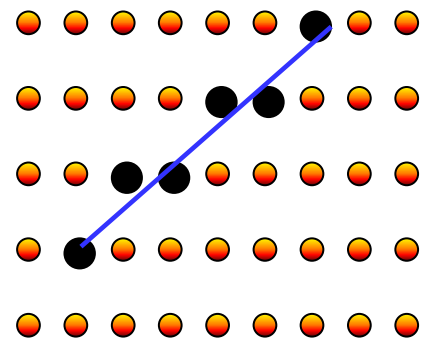
$$x_2 = 2, y_2 = 0.6 \times 2 + 0.4 = 1.6, \text{ draw } (2,2)$$

$$x_3 = 3, y_3 = 0.6 \times 3 + 0.4 = 2.2, \text{ draw } (3,2)$$

$$x_4 = 4, y_4 = 0.6 \times 4 + 0.4 = 2.8, \text{ draw } (4,3)$$

$$x_5 = 5, y_5 = 0.6 \times 5 + 0.4 = 3.4, \text{ draw } (5,3)$$

$$x_6 = 6, y_6 = 0.6 \times 6 + 0.4 = 4, \text{ draw } (6,4)$$



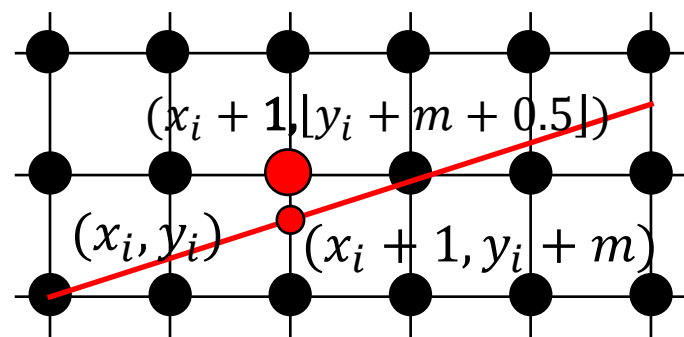
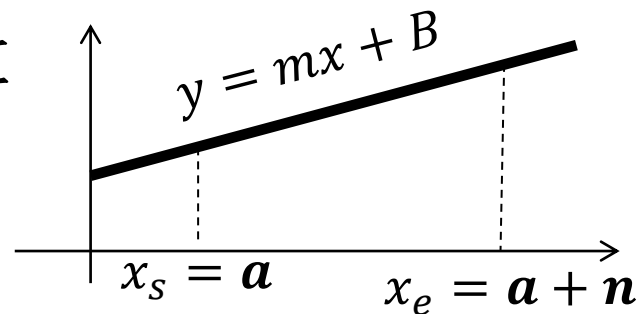
2 DDA算法(Digital differential analyzer)

- 平凡算法的缺点
 - 多次乘法运算，取整运算
- 注意到 x_i 每次增加1

$$x_{i+1} = x_i + 1.$$

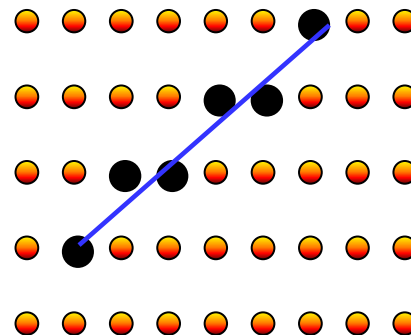
y_{i+1} 可以简化为：

$$\begin{aligned}
 y_{i+1} &= mx_{i+1} + B \\
 &= m(x_i + 1) + B \\
 &= m + (mx_i + B) \\
 &= y_i + m.
 \end{aligned}$$



DDA画线算法伪代码

```
void lineDDA(int x0, int x1, int y0, int y1){ //可处理任意斜率
    int dx = x1-x0, dy = y1-y0, step, k;
    float x_inc, y_inc, x = x0, y = y0;
    if (|dx| > |dy|) //斜率绝对值小于1
        step = dx;
    else //斜率绝对值大于1
        step = dy;
    x_inc = (float)dx/step; y_inc = (float)dy/step;
    glPointSize(3); glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_POINTS);
        for(k = 0; k <= step; k++){
            glVertex2i(int(x+0.5), int(y+0.5)); //画像素
            x += x_inc; y += y_inc;
        }
    glEnd();
}
```



例：DDA算法画线段(1,1) – (6,4)

- 线段方程

$$y = 0.6x + 0.4, x = 1, 2, 3, 4, 5, 6$$

- 初始化: $dx = 5, dy = 3, step = 5, x_{inc} = 1, y_{inc} = 0.6$

$x_1 = 1, y_1 = 1, \text{ draw } (1,1)$

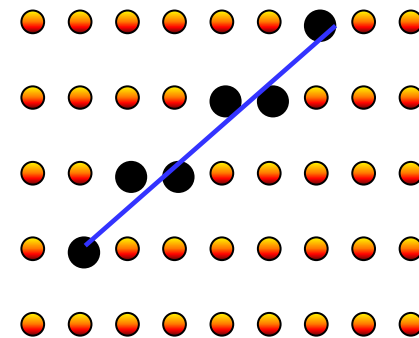
$x_2 = 2, y_2 = y_1 + 0.6 = 1.6, \text{ draw } (2,2)$

$x_3 = 3, y_3 = y_2 + 0.6 = 2.2, \text{ draw } (3,2)$

$x_4 = 4, y_4 = y_3 + 0.6 = 2.8, \text{ draw } (4,3)$

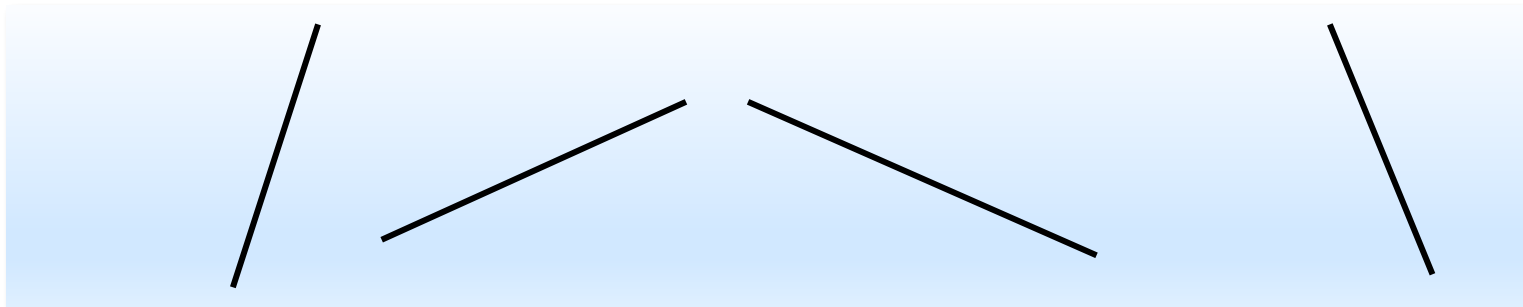
$x_5 = 5, y_5 = y_4 + 0.6 = 3.4, \text{ draw } (5,3)$

$x_6 = 6, y_6 = y_5 + 0.6 = 4, \text{ draw } (6,4)$



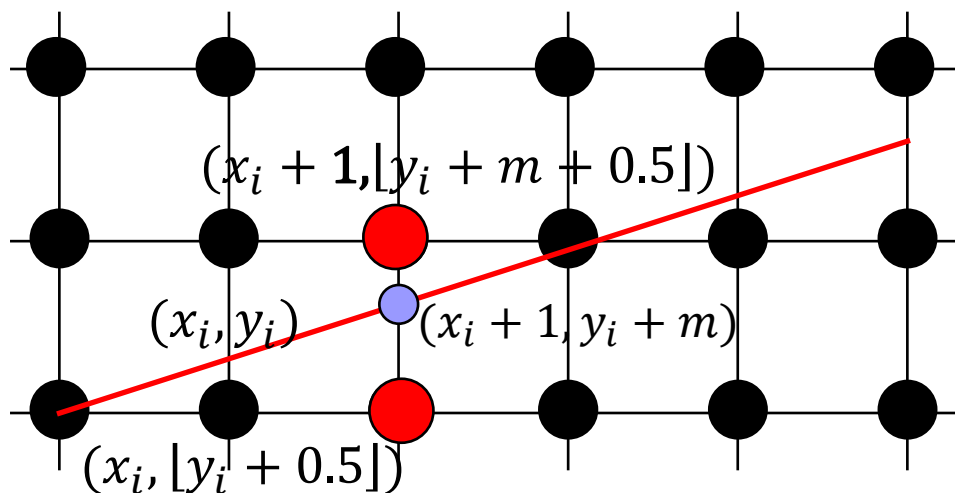
一般情形处理

- 伪代码中考虑了任意情斜率线段的光栅化 ($|m| \leq 1$, and $|m| > 1$)
- 斜率绝对值大于1时, 每次y坐标增加1

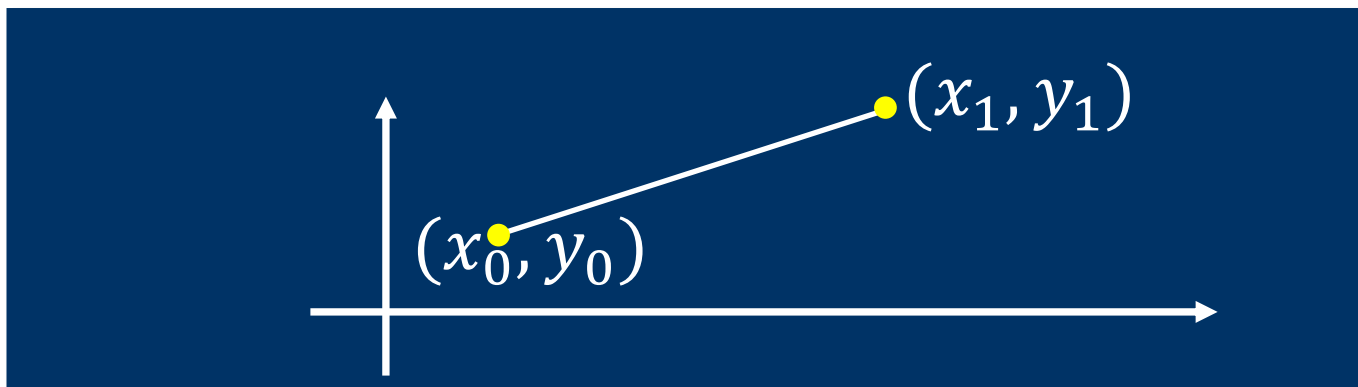


3 Bresenham算法

- DDA 利用了 y_i 的结果来计算 y_{i+1} , 减少1次乘积;
 - 仍然需要取整
- Bresenham算法: 下一次可选点的约束
 - 当前像素一旦确定, 下一个点是有限制的, 只能从2个中点选一个
 - 算法只涉及
 - $+$, $-$, 移位
 - 用于绘图仪、显卡等硬件实现



- 线段的两个端点: 从 (x_0, y_0) 到 (x_1, y_1)
 - 不妨设 $\Delta x = x_1 - x_0 > 0$, $\Delta y = y_1 - y_0 > 0$,
 - 记斜率 $m = \frac{\Delta y}{\Delta x}$, 且约定 $m < 1$
 - 线段上点 (x_i, y_i) 对应的像素点为 $(\bar{x}_i, \bar{y}_i)$



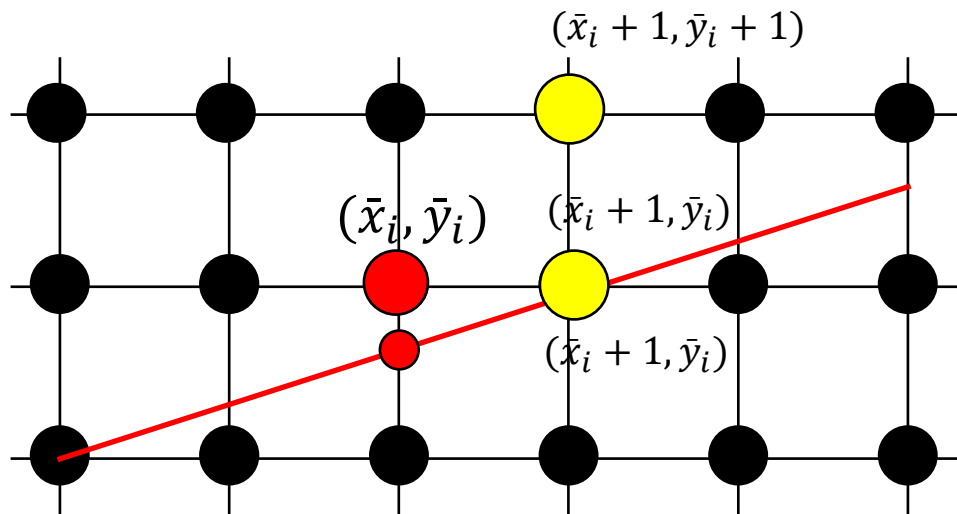
Bresenham的观察: 缩小可选点范围

● Bresenham 算法

- 也是从 $x = x_0$ 出发, 每次增加1
- 记选中的第 i 个像素点为 $(\bar{x}_i, \bar{y}_i)$
- 第 $i + 1$ 个像素点只能从下面2个点中的一个

$(\bar{x}_i + 1, \bar{y}_i), (\bar{x}_i + 1, \bar{y}_i + 1)$

● 怎么选?

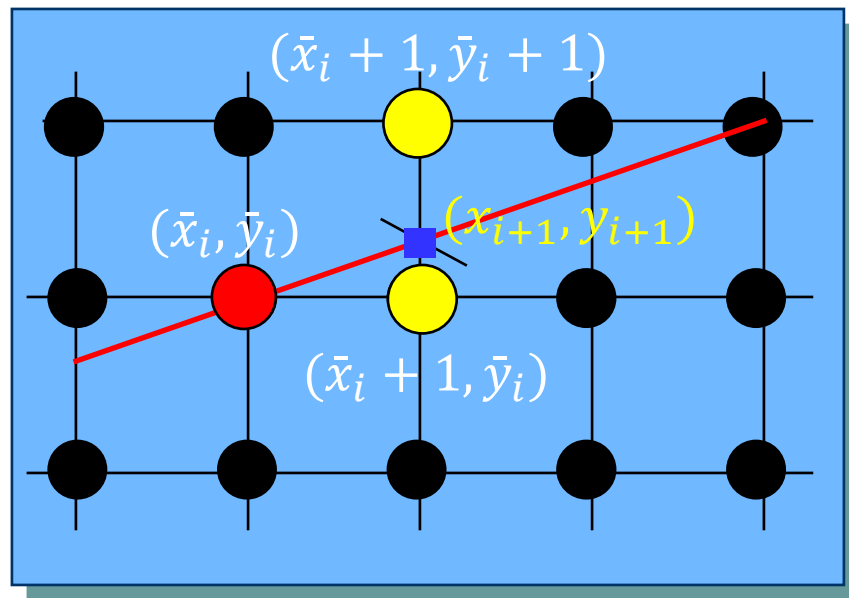


选点判定准则 (Criteria)

- 选择离直线较近的那个像素点, 也是离 (x_{i+1}, y_{i+1}) 较近的点

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = mx_{i+1} + B$$
$$= mx_i + m + B$$

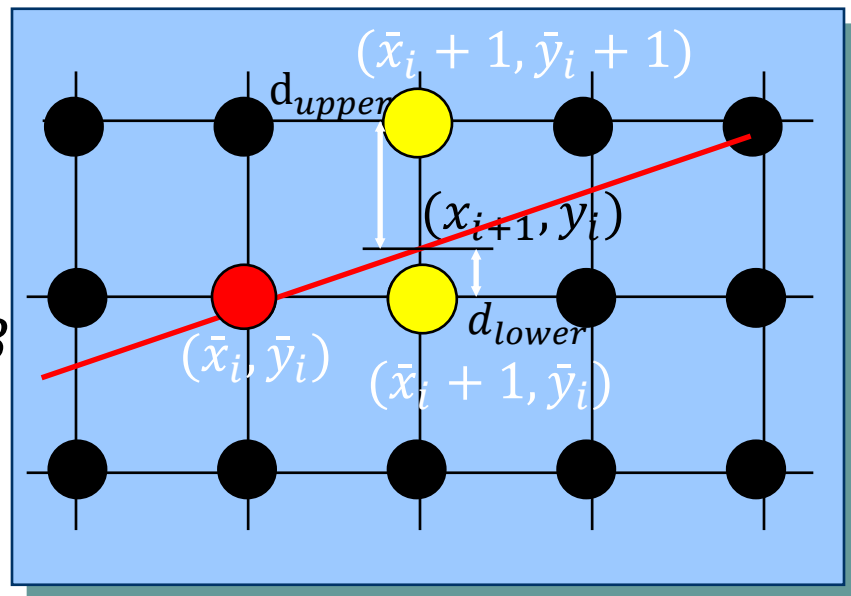


判别准则计算

- 两个像素到 (x_{i+1}, y_{i+1}) 的距离分别为：

$$\begin{aligned} d_{upper} &= \bar{y}_i + 1 - y_{i+1} \\ &= \bar{y}_i + 1 - mx_{i+1} - B \end{aligned}$$

$$\begin{aligned} d_{lower} &= y_{i+1} - \bar{y}_i \\ &= mx_{i+1} + B - \bar{y}_i \end{aligned}$$



- 判定：
- 1) $d_{lower} - d_{upper} > 0$ 取右上方点；
 - 2) $d_{lower} - d_{upper} < 0$ 取右边点；
 - 3) $d_{lower} - d_{upper} = 0$ 可任取，如取右边点。

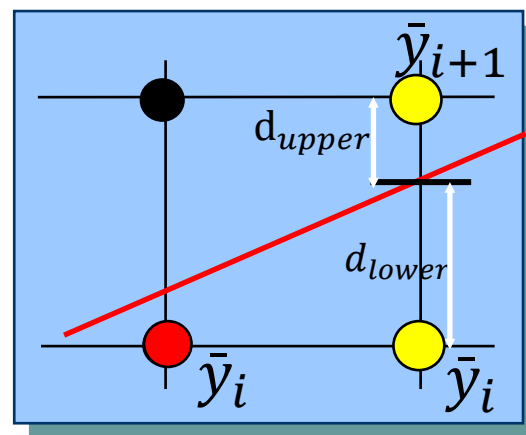
判别标准化简

$$\begin{aligned}
 d_{lower} - d_{upper} &= m(x_i + 1) + B - \bar{y}_i \\
 &\quad - (\bar{y}_i + 1 - m(x_i + 1) - B) \\
 &= 2m(x_i + 1) - 2\bar{y}_i + 2B - 1
 \end{aligned}$$

上式与下式符号相同

重要

$$\begin{aligned}
 p_i &= \Delta \hat{x} \cdot (d_{lower} - d_{upper}) \\
 &= 2\Delta y \cdot (x_i + 1) - 2\Delta x \cdot \bar{y}_i + (2B - 1)\Delta x \\
 &= 2\Delta y \cdot x_i - 2\Delta x \cdot \bar{y}_i + (2B - 1)\Delta x + 2\Delta y \\
 &= 2\Delta y \cdot x_i - 2\Delta x \cdot \bar{y}_i + c
 \end{aligned}$$



注:

$$\begin{aligned}
 \Delta x &= x_1 - x_0, \Delta y = y_1 - y_0, m = \Delta y / \Delta x \\
 c &= (2B - 1)\Delta x + 2\Delta y
 \end{aligned}$$

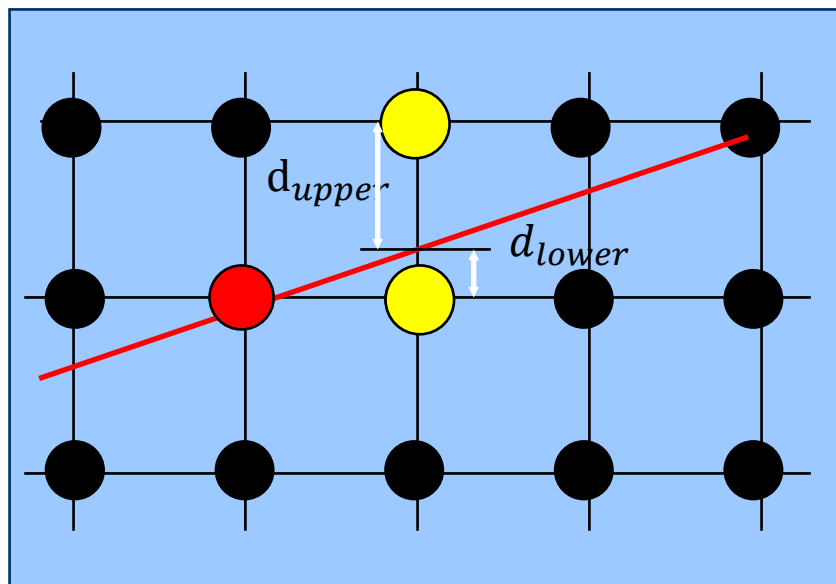
p_i 的迭代公式

if $p_i > 0$ **then** $(x_i + 1, \bar{y}_i + 1)$ **is selected**

if $p_i < 0$ **then** $(x_i + 1, \bar{y}_i)$ **is selected**

if $p_i = 0$ **one of the above two**

虽然 p_i 都是整数运算，但仍然需要2次乘积，2次加减



判别式初始化： p_0 的计算

已知 (x_0, y_0) 计算 p_0 ，用于判断 p_1 的取值：

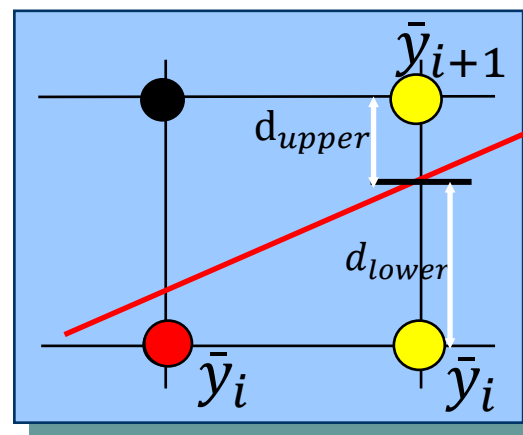
$$\Delta x \bar{y}_0 = \Delta y x_0 + B \Delta x$$

$$\begin{aligned} p_0 &= 2\Delta y x_0 - 2\Delta x \bar{y}_0 + (2B - 1)\Delta x + 2\Delta y \\ &= 2\Delta y x_0 - 2(\Delta y x_0 + B \Delta x) + (2B - 1)\Delta x + 2\Delta y \\ &= 2\Delta y - \Delta x \end{aligned}$$

$$\therefore p_0 = 2\Delta y - \Delta x$$

$$p_i = 2\Delta y x_i - 2\Delta x \bar{y}_i + (2B - 1)\Delta x + 2\Delta y$$

$$(x_0, \bar{y}_0): \bar{y}_0 = \frac{\Delta y}{\Delta x} x_0 + B$$



p_i 的迭代计算 (p_i 用以判断 $\bar{y}_{i+1}$ 的取值)

- 由 p_i 计算 p_{i+1}

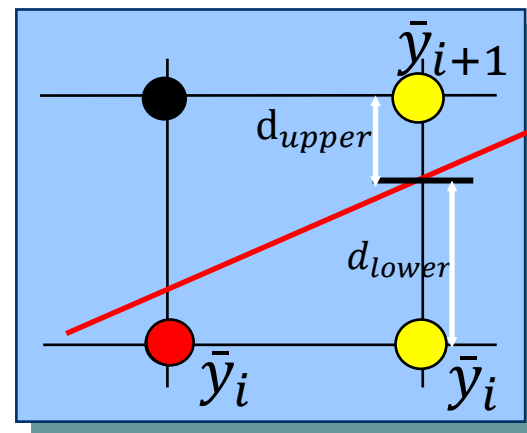
$$\begin{aligned} p_{i+1} - p_i &= (2\Delta y \cdot x_{i+1} - 2\Delta x \cdot \bar{y}_{i+1} + c) \\ &\quad - (2\Delta y \cdot x_i - 2\Delta x \cdot \bar{y}_i + c) \\ &= 2\Delta y - 2\Delta x(\bar{y}_{i+1} - \bar{y}_i) \end{aligned}$$

- 如果 $p_i \leq 0$ 那么 $\bar{y}_{i+1} = \bar{y}_i$

$$\therefore p_{i+1} = p_i + 2\Delta y$$

- If $p_i > 0$ then $\bar{y}_{i+1} = \bar{y}_i + 1$

$$\therefore p_{i+1} = p_i + 2\Delta y - 2\Delta x$$



$$p_i = 2\Delta y x_i - 2\Delta x \bar{y}_i + (2B - 1)\Delta x + 2\Delta y$$

Bresenham算法框架

1) 画 (x_0, y_0)

计算 $\Delta x, \Delta y, a = 2\Delta y, b = 2\Delta y - 2\Delta x,$

$$p_0 = 2\Delta y - \Delta x$$

2) 如果 $p_i \leq 0$ 画 $(x_i + 1, \bar{y}_i)$

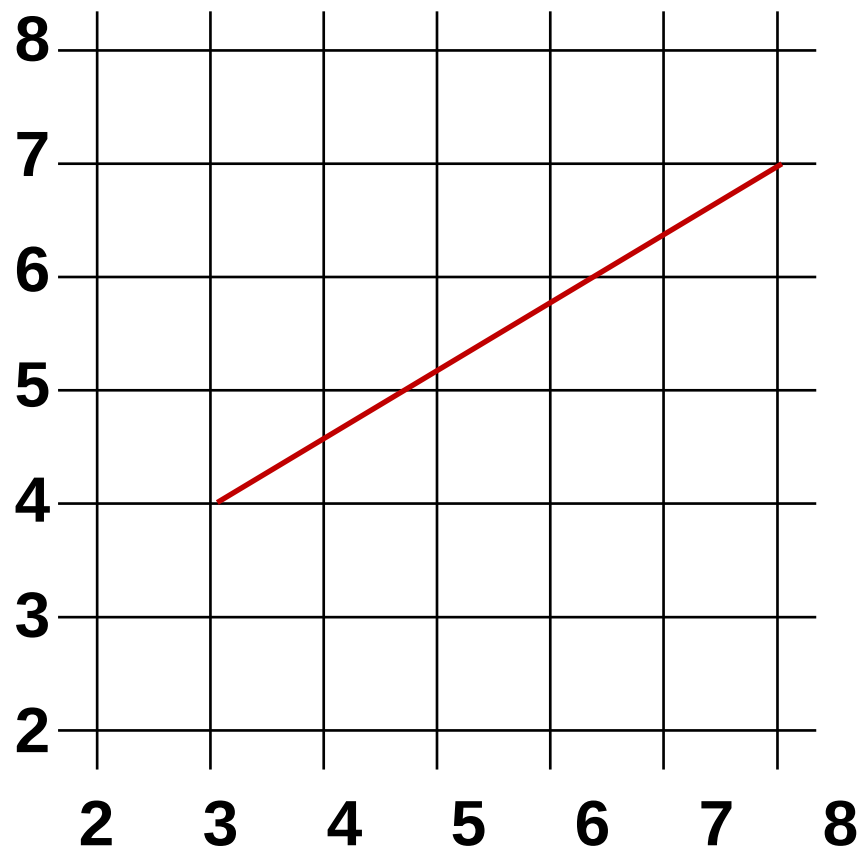
$$\text{计算 } p_{i+1} = p_i + a$$

否则, 如果 $p_i > 0$ 画 $(x_i + 1, \bar{y}_i + 1)$

$$\text{计算 } p_{i+1} = p_i + b$$

3) 重复2) 直到 x_i 等于右端点 x 坐标

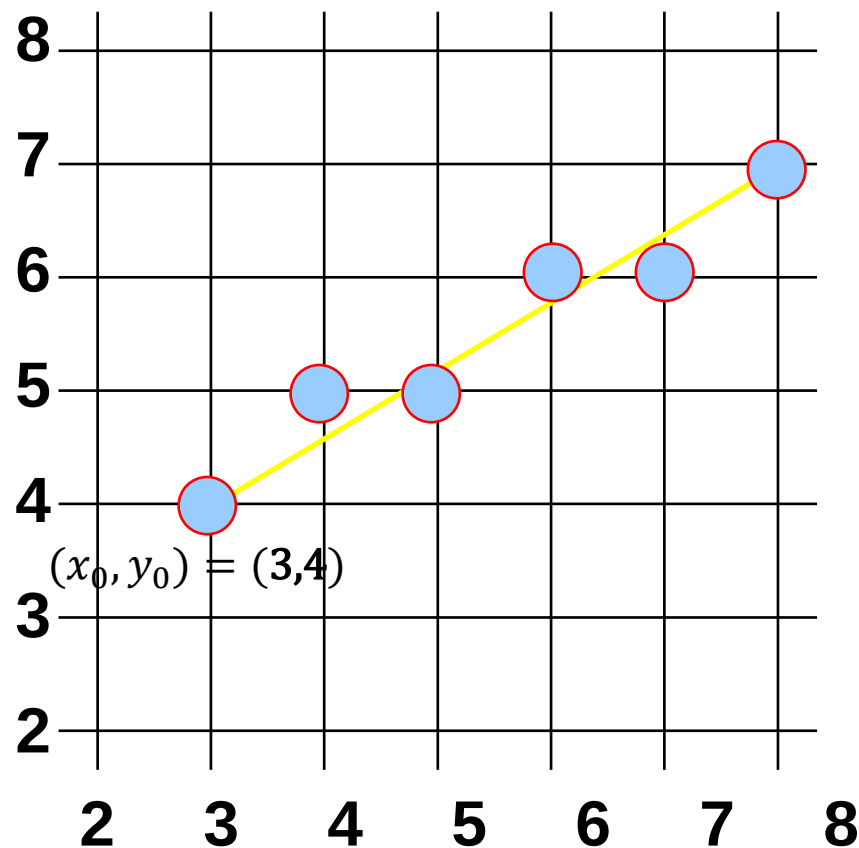
例：用Bresenham算法画线段 $(3, 4) - (8, 7)$



(Continued)



k	p_k	$(\bar{x}_{k+1}, \bar{y}_{k+1})$
0	1	(4,5)
1	-3	(5,5)
2	3	(6,6)
3	-1	(7,6)
4	5	(8,7)



注: $p_0 = 2\Delta y - \Delta x$ $p_{i+1} = p_i + 2\Delta y$ $p_{i+1} = p_i + 2\Delta y - 2\Delta x$

Bresenham算法伪代码

/* Bresenham algorithm for drawing line segments.

Suppose the slope holds $(0 \leq m \leq 1)$ */

```
void lineBresenham(int x0,int y0,int x1,int y1)
```

```
{
```

```
    int temp;
```

```
    if (x1 < x0){ // Swap the endpoints
```

```
        temp = x0; x0 = x1; x1 = temp;
```

```
        temp = y0; y0 = y1; y1 = temp;
```

```
}
```

```
int k, dx = x1-x0 , dy = y1-y0;
int Twody = 2*dy, Twody_Twodx = 2*dy-2*dx, p= Twody-dx;
glPointSize(5); glColor3f(1,0,0);
glBegin(GL_POINTS);
for (k = 0; k <= dx; k++){
    glVertex2i(x+k, y);
    if(p <= 0)
        p += Twody; // Draw the right pixel
    else {
        y++; // Draw the upper right pixel
        p += Twody_Twodx;
    }
}
glEnd()
}
```

一般情形讨论

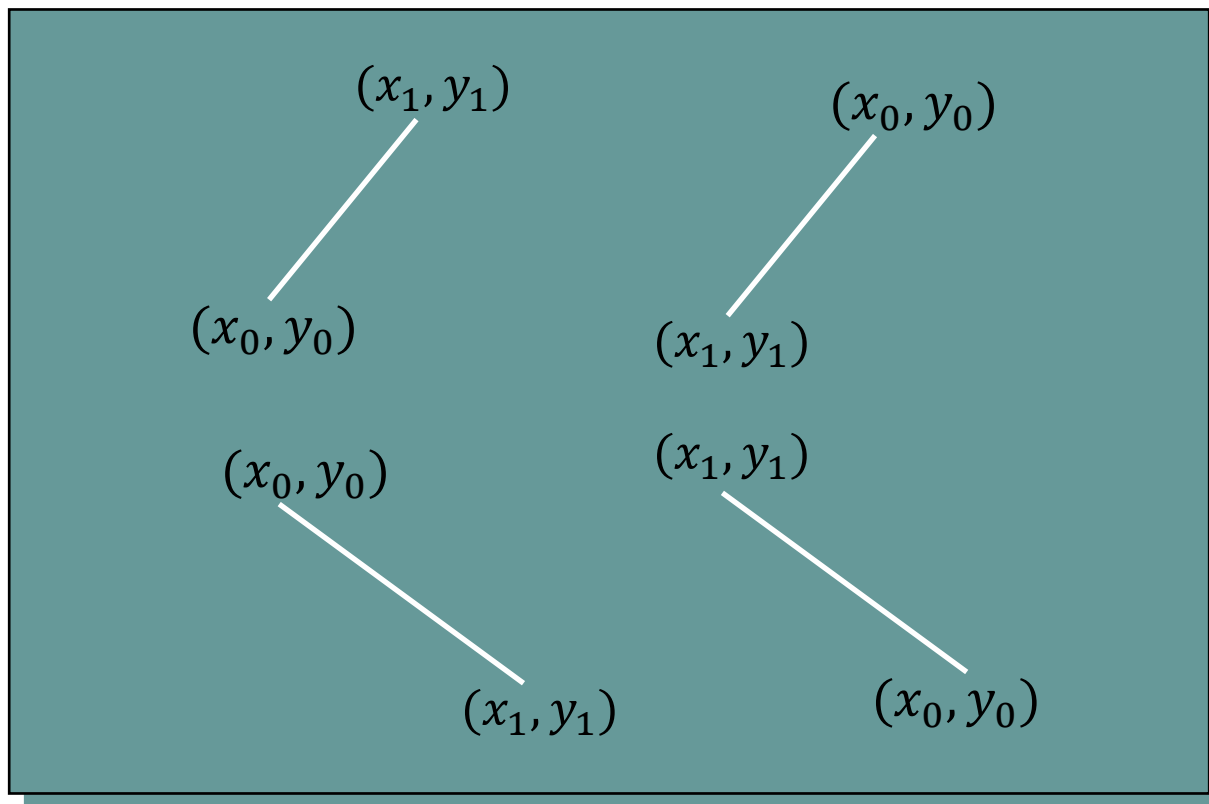
- Bresenham算法只涉及整数加、减和移位计算，易于硬件实现
- 伪代码只处理了斜率在 $[0, 1]$ 间情形

(x_0, y_0) ——— (x_1, y_1) (x_1, y_1) ——— (x_0, y_0)

- 剩下斜率小于0和大于1的情况

(x_1, y_1) (x_0, y_0) (x_1, y_1)
 (x_0, y_0) (x_1, y_1) (x_0, y_0)

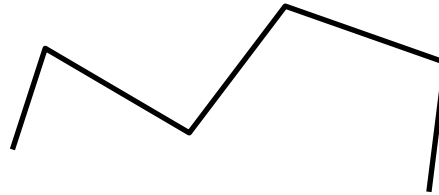
- 没有讨论斜率绝对值大于1: ($|m| > 1$)



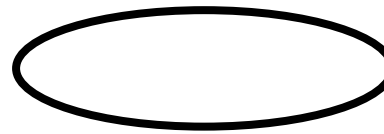
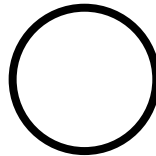
Other problems

- The coordinates of endpoints are not an integer

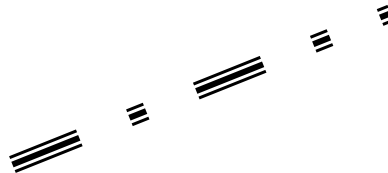
- Polyline



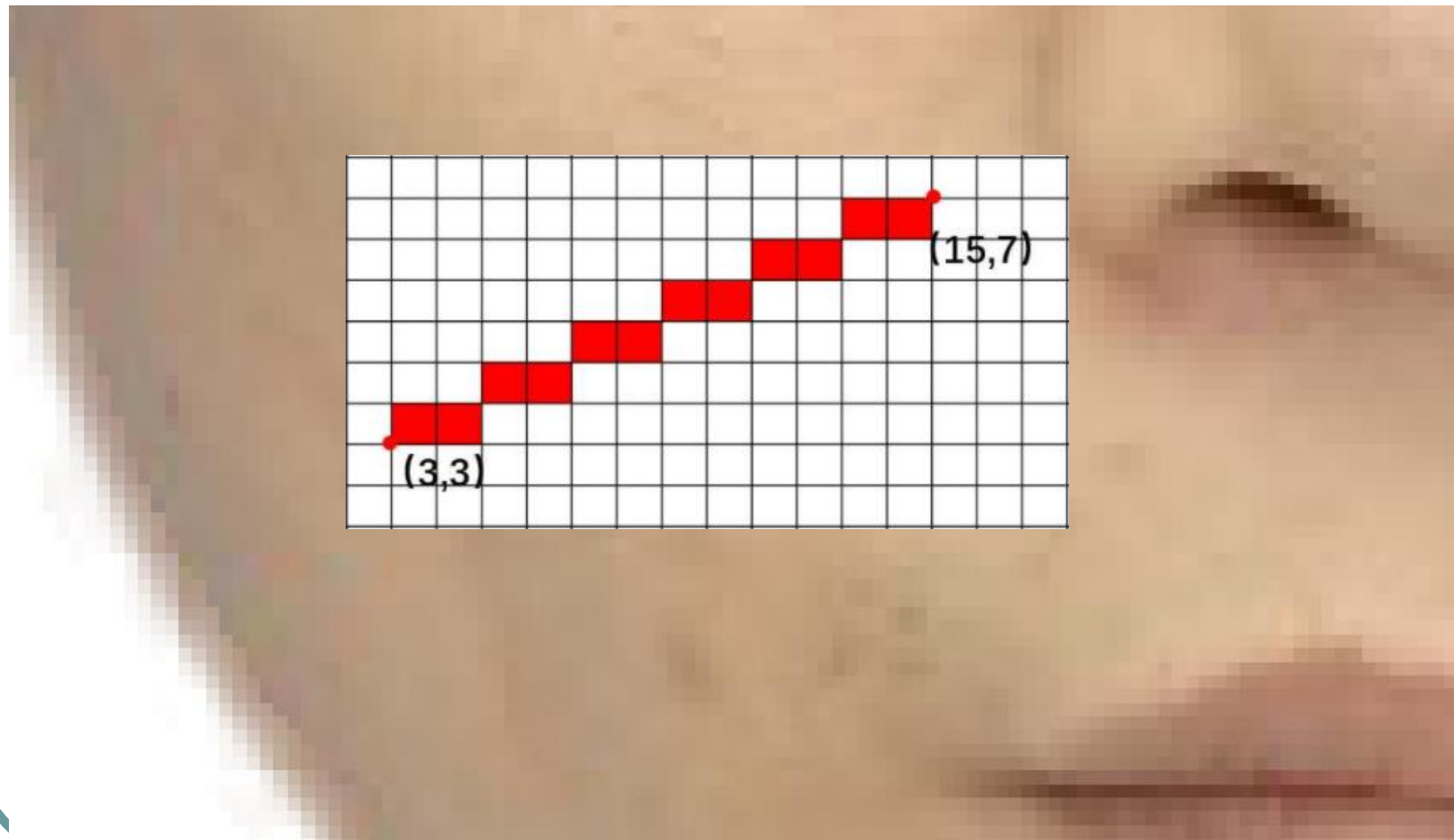
- Other primitives: circles, ellipsoids



- Line pattern and thickness?



Antialiasing(走样与反走样)



Dr. Jack Elton Bresenham

- 1937, 出生
- 1962, 在IBM任职时提出Bresenham算法, 1965年发表
- 1964, 获得Stanford 大学博士学位



Agenda



- Scan conversion of line segments
 - DDA algorithm
 - Bresenham algorithm
- 多边形扫描转换
(Scan conversion of polygon)
 - 区域填充 (Fill-area algorithms)
 - 多边形扫描线填充算法 (Scan-line polygon-fill algorithm)

(Chap 4, sections 4-10, 4-13)

光栅图形

- 光栅图形的本质

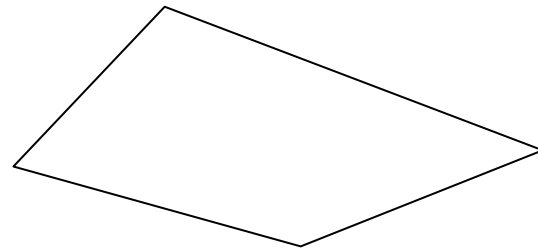
- 点阵表示

- 线框图

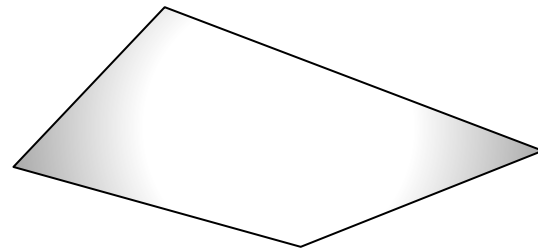
- 先扫描转换

- 真实感图形

- 面着色
- 画面明暗自然、色彩丰富；
- 比线框图更生动、直观、真实

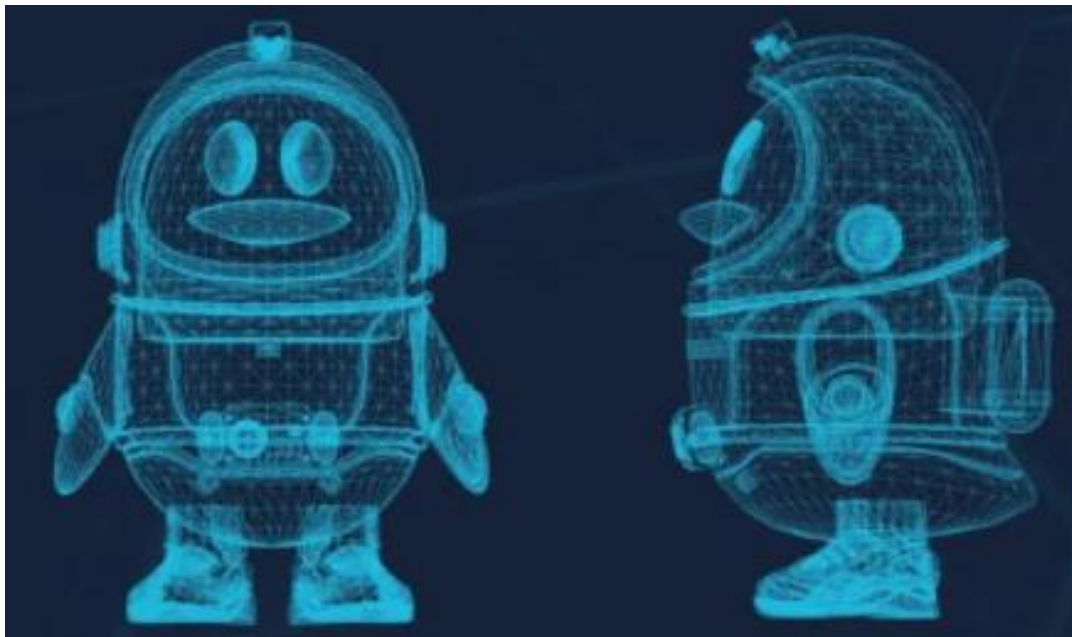


线框平面多边形



着色的平面多边形

光栅图形



线框多边形物体：
只需扫描转换线段



填充多边形物体：
要扫描转换多边形

Hair Meshes

Cem Yuksel
Cyber Radiance

Scott Schaefer
Texas A&M University

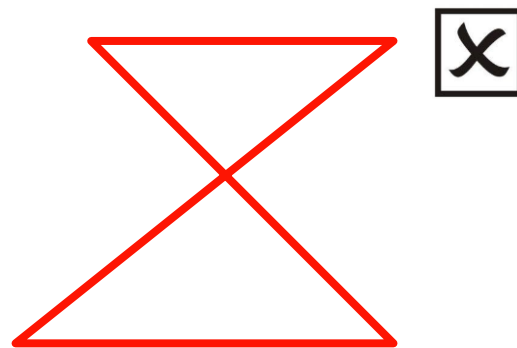
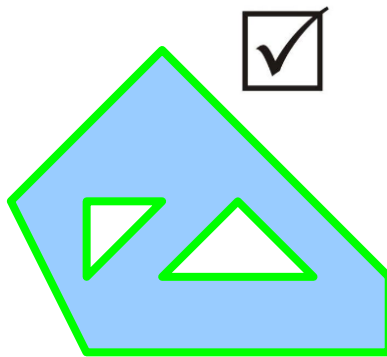
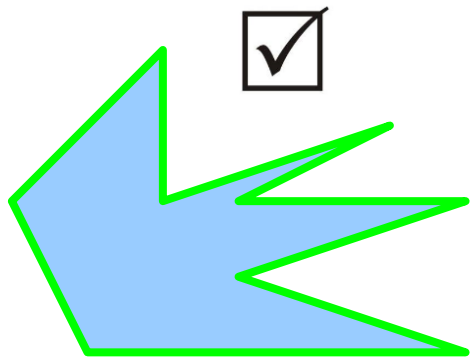
John Keyser
Texas A&M University

SIGGRAPH Asia 2009

光栅图形的基本概念

- 简单多边形 (Simple Polygons)

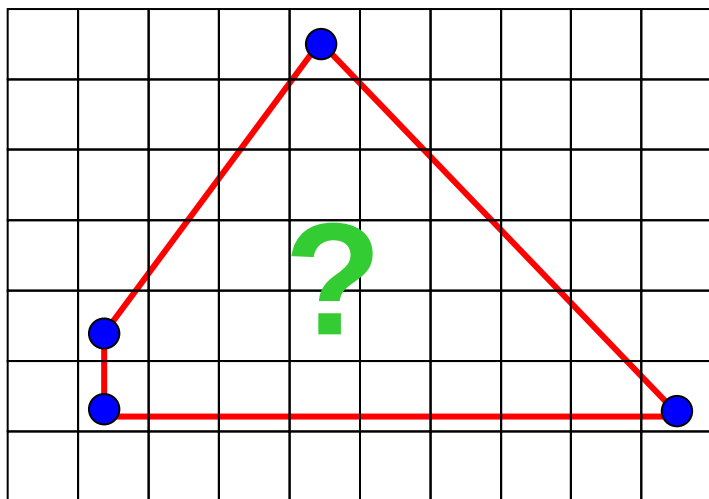
- 无自相交



- 图形学中多边形的两种表示方式

- 顶点表示：多边形的有序顶点序列
- 点阵表示：多边形内部的像素集合

顶点表示(对象表示)



多边形的顶点表示

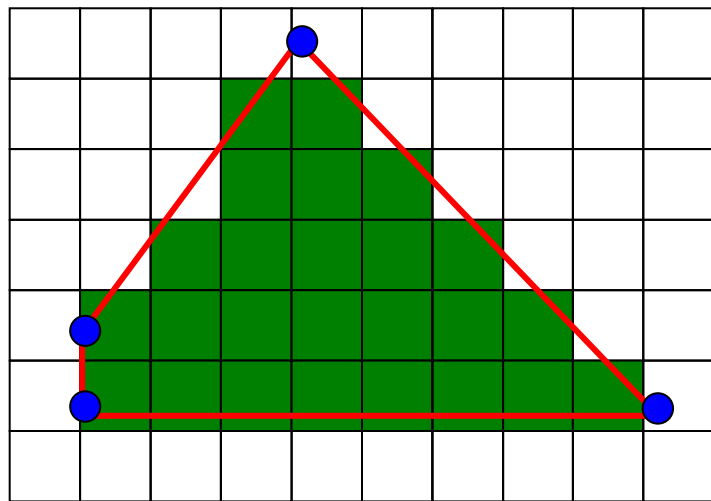
- 优点

- 直观, 有几何意义
- 便于几何变换
- 存储量小

- 不足

- 不能直接用于多边形着色

点阵表示(图象表示)

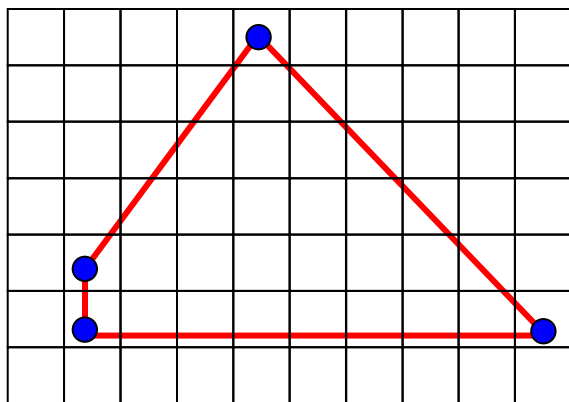


多边形的点阵表示

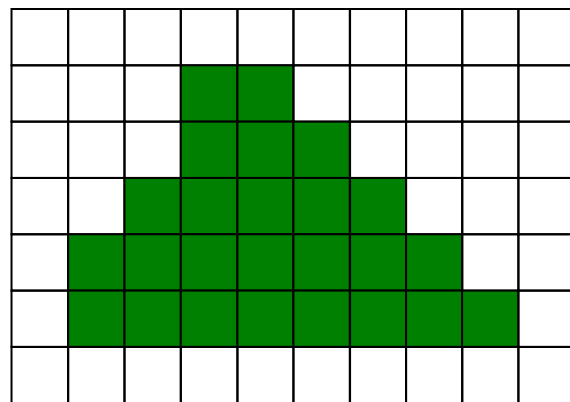
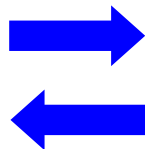
- 优点
 - 可用帧缓冲器(frame buffer)表示图形
 - 支持面着色(rendering)
- 缺点
 - 无几何信息
 - 存储量大

多边形扫描转换

- 多边形的扫描转换：顶点表示 → 点阵表示
 - 从多边形的给定边界出发，求出其内部的各个像素
 - 并给帧缓冲器中各个对应元素设置相应灰度或颜色



多边形的顶点表示



多边形的点阵表示

主要内容

- 基本概念
- 区域填充(8.10.4,flood filling)
 - 四连通区域和八连通区域
 - 连通区域的种子填充算法
- 多边形的扫描转换
- 多边形的扫描转换与区域填充的比较

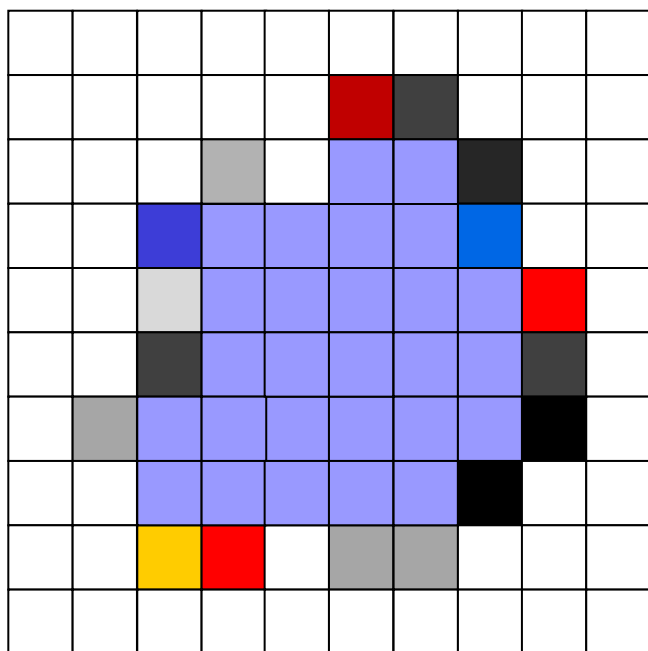
Agenda



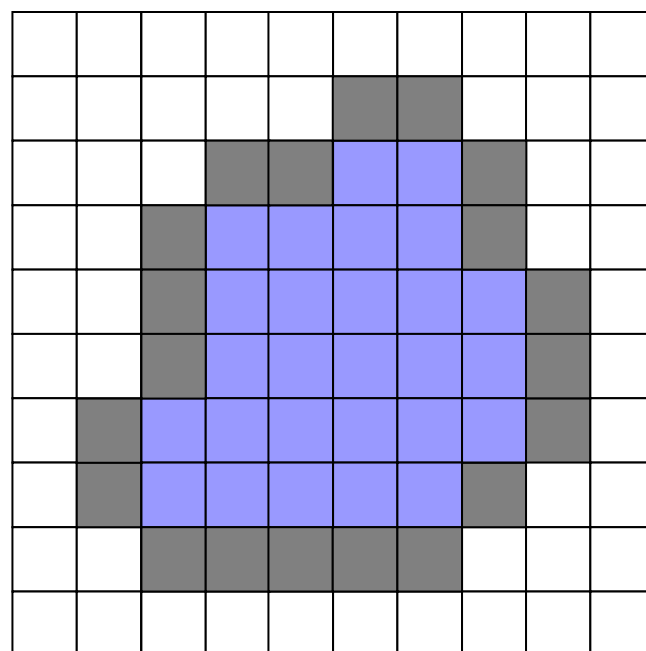
- Scan conversion of line segments
 - DDA algorithm
 - Bresenham algorithm
 - Scan conversion of polygon
 - Fill-area algorithms(区域填充)
 - Scan-line polygon-fill algorithm
- (Chap 4, sections 4-10)

多边形区域的点阵表示

● 内部表示

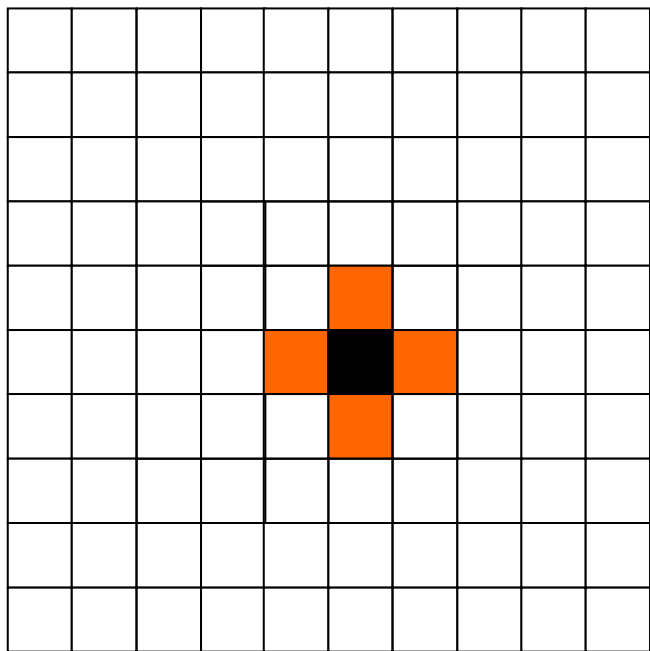


● 边界表示



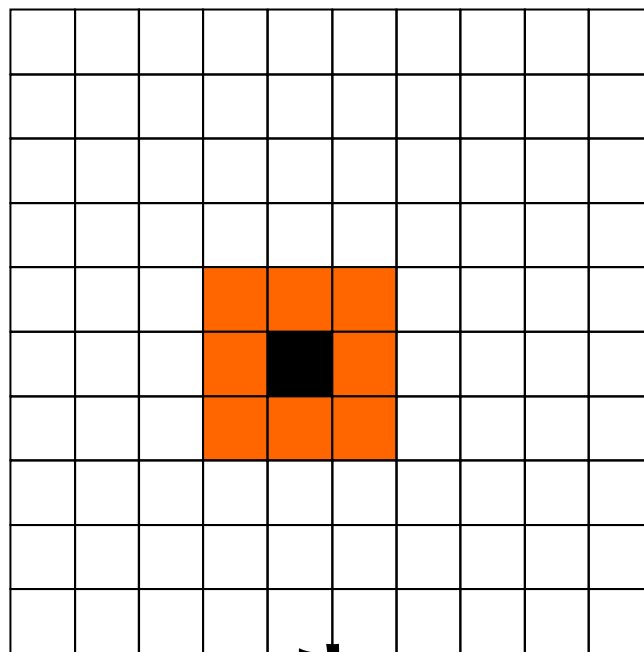
邻接关系

- 四连通邻域



一个像素有**四**个相邻像素

- 八连通邻域

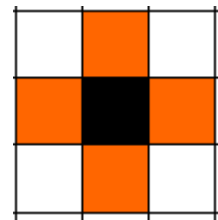


一个像素有**八**个相邻像素

区域的连通类型-定义

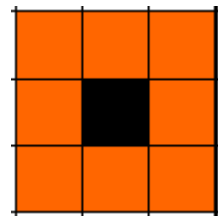
● 四连通区域

- 区域内任两个像素，从一个出发，可以在区域内部通过上、下、左、右四个方向移动，到达另一个

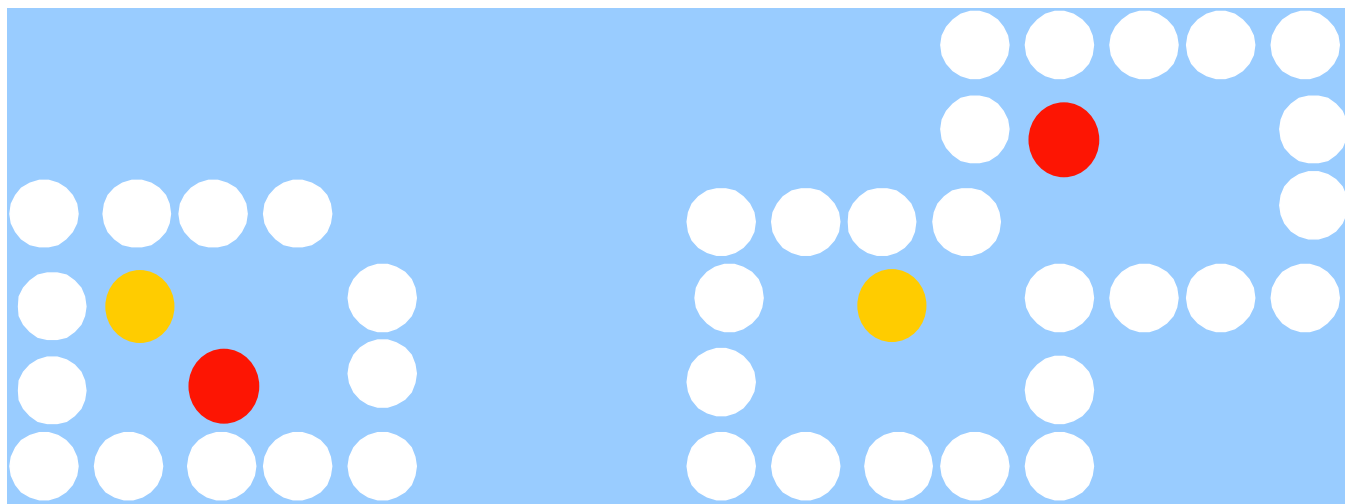


● 八连通区域

- 区域内任两个像素，从一个出发，可以在区域内部通过水平、垂直、正对角线、反对角线八个方向移动到达另一个



区域的连通类型---演示

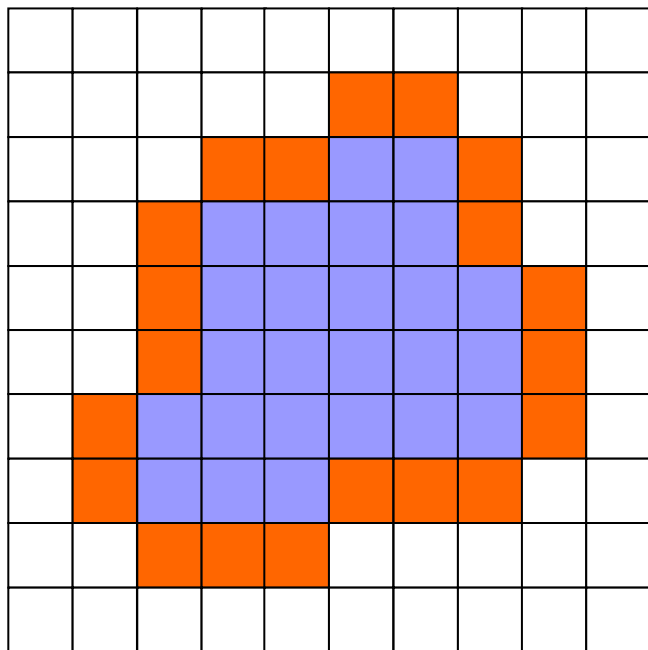


四连通区域

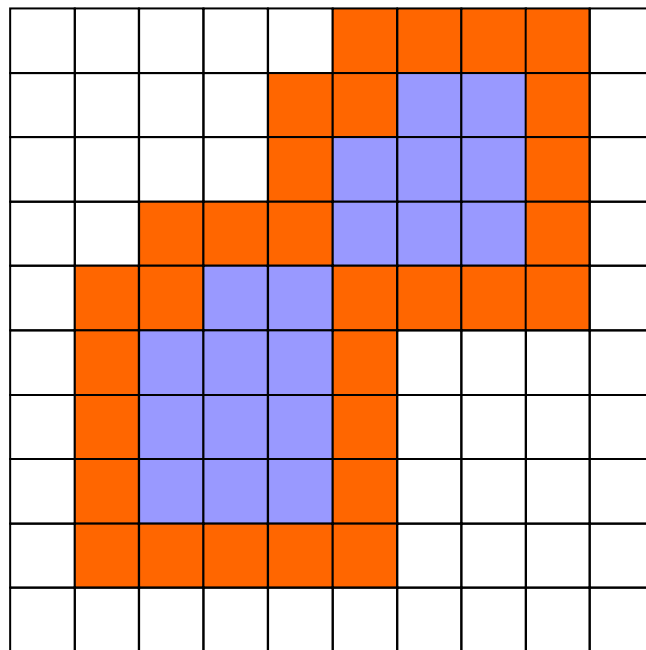
八连通区域

区域的连通类型—例子

● 四连通区域实例



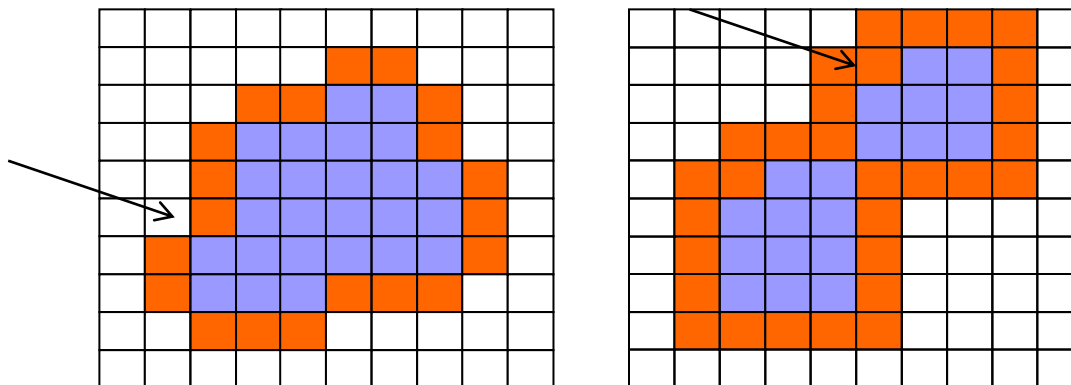
● 八连通区域实例



区域的类型

● 四连通和八连通区域的关系

- 四连通区域 $\subseteq$ 八连通区域 (反之不成立)
- 四连通区域的边界是八连通区域
- 八连通区域的边界是四连通区域



内部表示区域种子填充

- 内部表示区域(记为G),
 - 像素原来颜色为G0
 - 需填充的颜色为G1
 - 任给G内部的一个种子点 (x, y) 。

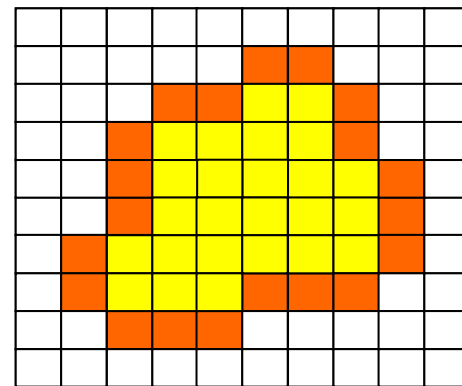


Paint brush: Fill with color

内部表示区域种子填充算法(四连通区域)

```
Flood_Fill_4(x, y, G0, G1)
{
    if(GetPixel(x,y) == G0 ){ // GetPixel(x,y)返回(x,y)的颜色
        SetPixel(x, y, G1); //将(x,y)的添上颜色G1
        Flood_Fill_4(x-1, y, G0, G1);
        Flood_Fill_4(x, y+1, G0, G1);
        Flood_Fill_4(x+1, y, G0, G1);
        Flood_Fill_4(x, y-1, G0, G1);
    }
}
```

```
void glReadPixels (GLint x, GLint y,
GLsizei width, GLsizei height, GLenum
format, GLenum type, GLvoid * data) ;
```



边界表示区域种子填充

- 问题: 边界表示区域为G,
 - 边界像素颜色为G0
 - 任给G内部的一个种子点(x,y)
 - 需填充的区域内部颜色为G1



Paint brush: Fill with color

边界表示区域种子填充算法

```
Fill_Boundary_4_Connected(x, y, G0, G1){  
    // (x,y) 种子像素的坐标;  
    // G0 边界像素颜色; G1 需要填充的内部像素颜色  
    if(GetPixel(x,y) != G0 && GetPixel(x,y) != G1){  
        // GetPixel(x,y): 返回像素(x,y)  
        SetPixel(x, y, G1); // 将像素(x, y)置成填充颜色  
        Fill_Boundary_4Conncted(x, y+1, G0, G1);  
        Fill_Boundary_4Conncted(x, y-1, G0, G1);  
        Fill_Boundary_4Conncted(x-1, y, G0, G1);  
        Fill_Boundary_4Conncted(x+1, y, G0, G1);  
    }  
}
```

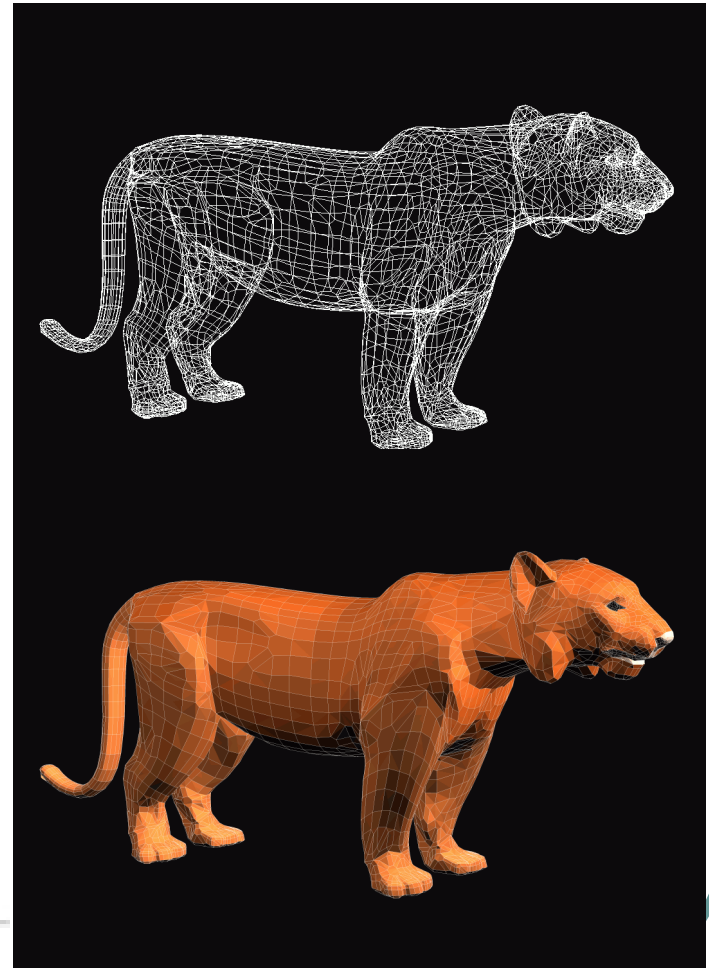
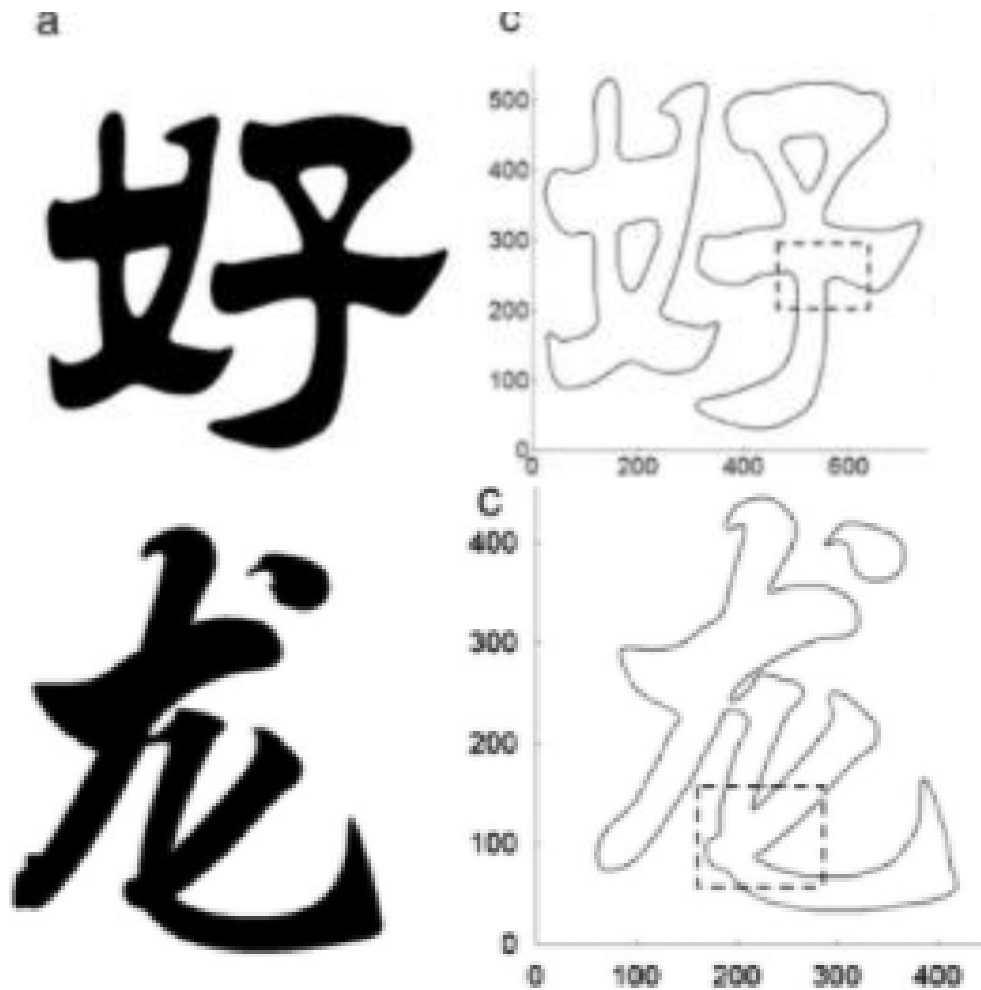
Agenda

- Scan conversion of line segments
 - DDA algorithm
 - Bresenham algorithm
- Scan conversion of polygon
 - Fill-area algorithms(区域填充)
 - Scan-line polygon-fill algorithm (多边形的扫描转换; Chap 8, section 8.11.3)

内容

- 基本概念
- 区域填充
- 多边形的扫描转换(Polygon rasterization/scan conversion)
 - 逐点判断算法(蛮力, brute force)
 - 扫描线算法
 - 连贯性概念: 区域、扫描线、边
 - 奇异点的处理
 - 算法的数据结构与实现
 - 三角形扫描转换
- 多边形的扫描转换与区域填充的比较

应用背景



逐点判断算法(蛮力方法)

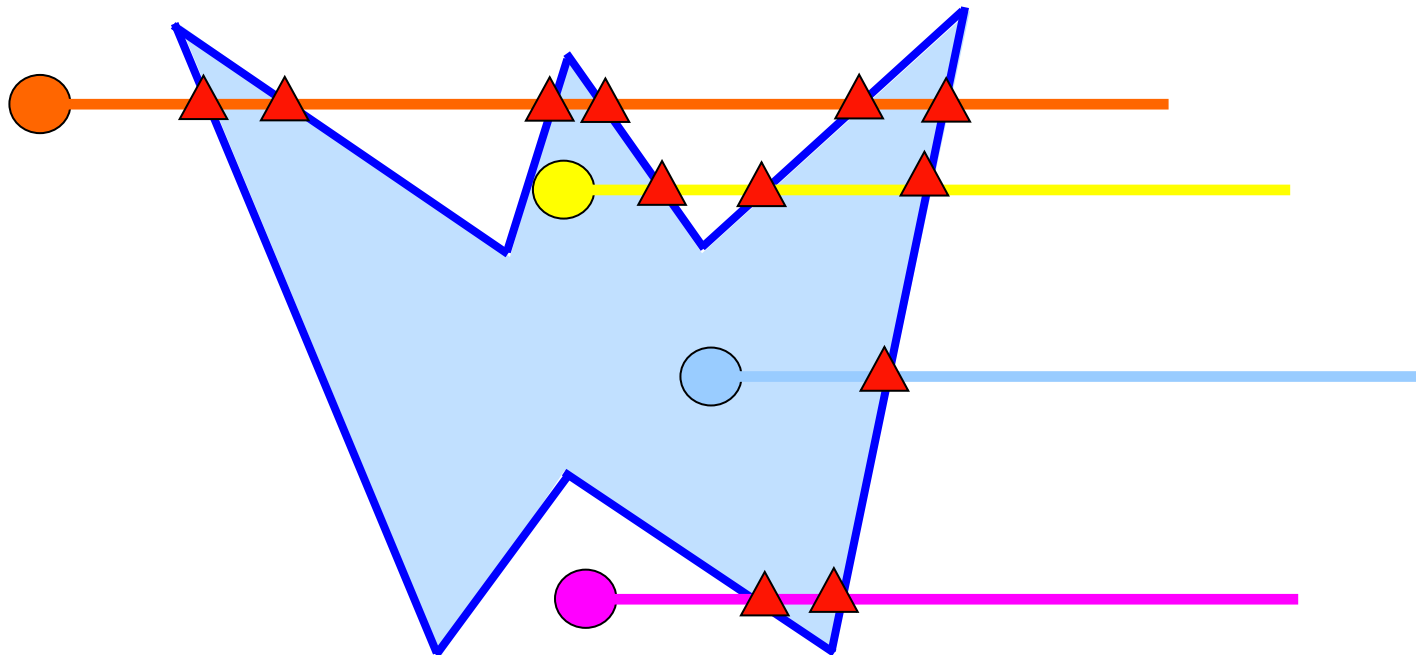
- 逐点判断

- 逐个像素判别其是否位于多边形内部

- 判断一个点是否位于多边形内部：射线法

- 从当前像素发射一条射线，计算射线与多边形的交点个数
 - 内部：奇数个交点
 - 外部：偶数个交点

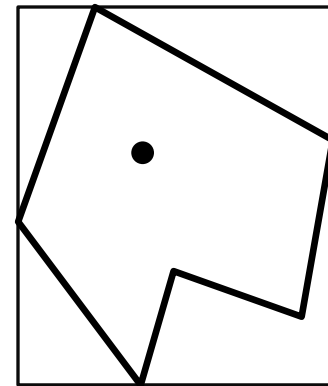
逐点判断算法



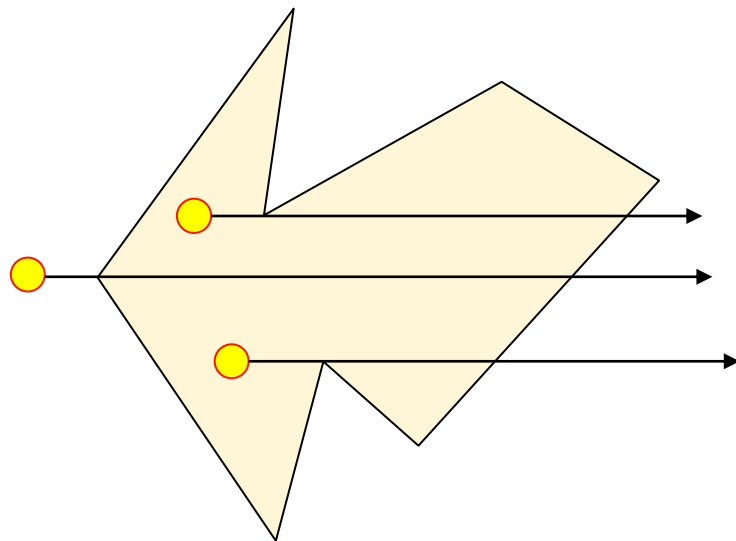
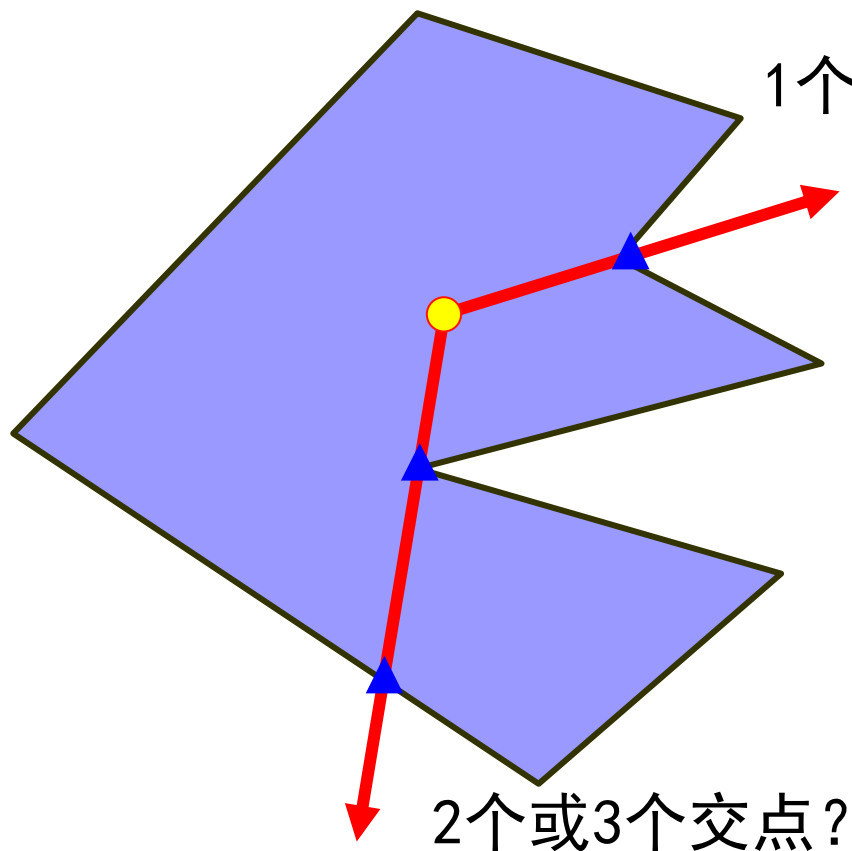
判断一点是否位于多边形内部？

逐点判断-区域填充蛮力算法

```
for(y=0; y<=y_resolution; y++)  
  for(x=0; x<=x_resolution; x++)  
  {  
    if(inside(polygon, x+0.5, y+0.5))  
      setpixel(framebuffer, x, y, polygon_color)  
    else  
      setpixel(framebuffer, x, y, background_color)  
  }
```

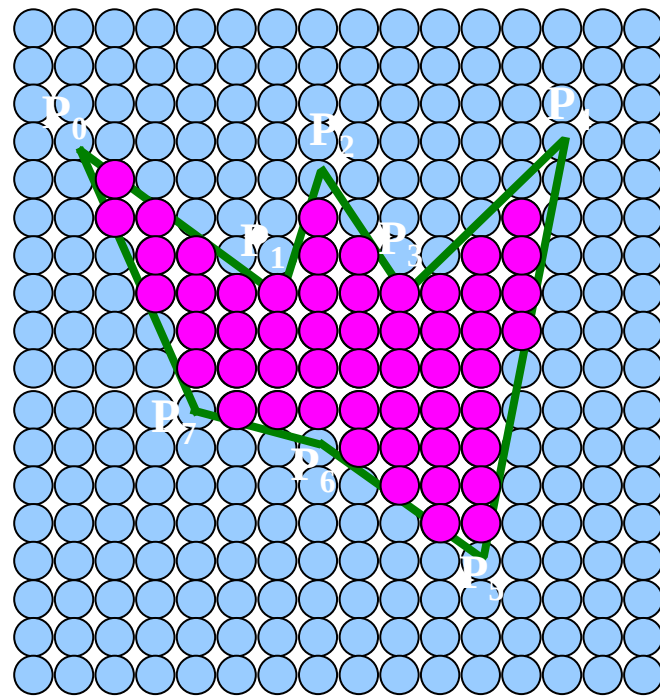


逐点判断算法中的奇异情况

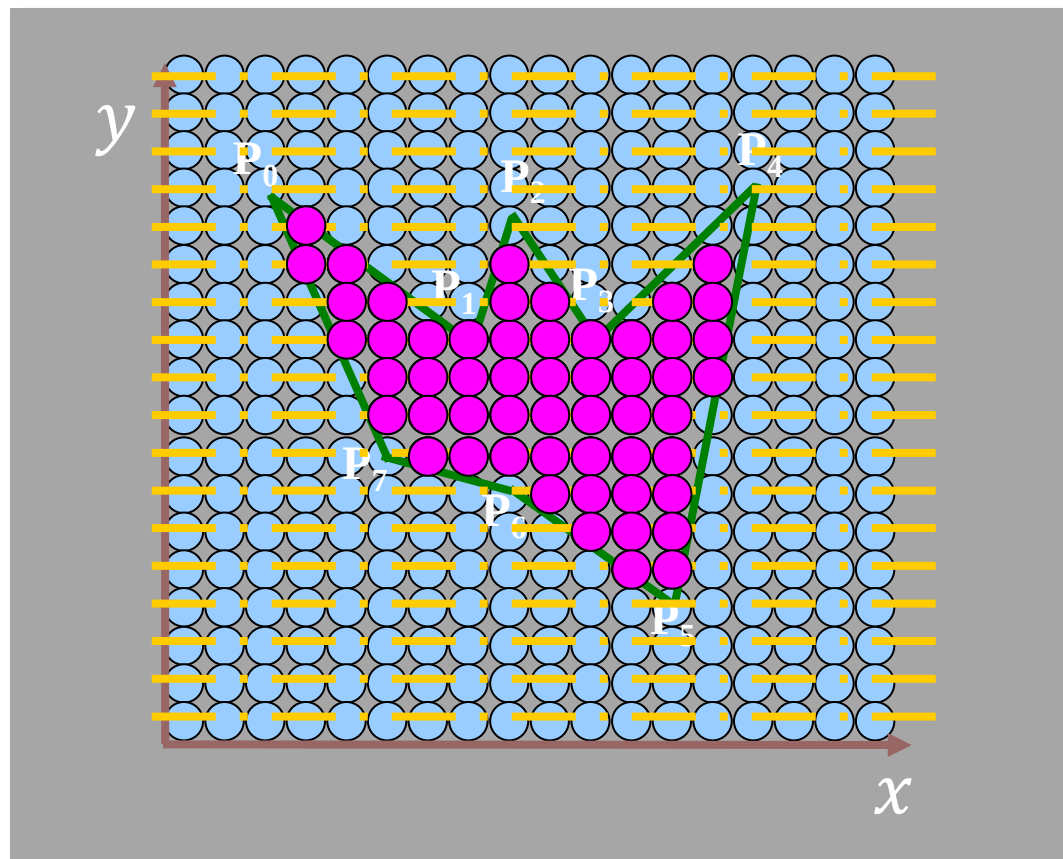


逐点判断算法的不足

- 速度慢
 - 几十万上百万像素多边形内外判断，需大量求交、乘除运算
- 没有利用像素间的联系
 - 相邻像素有共同属性
- 结论：逐点判断算法不可取！



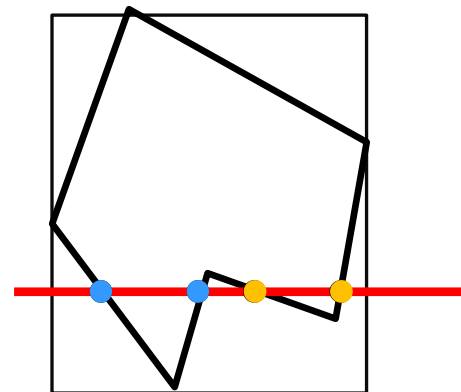
多边形扫描转换算法



扫描转换示意图

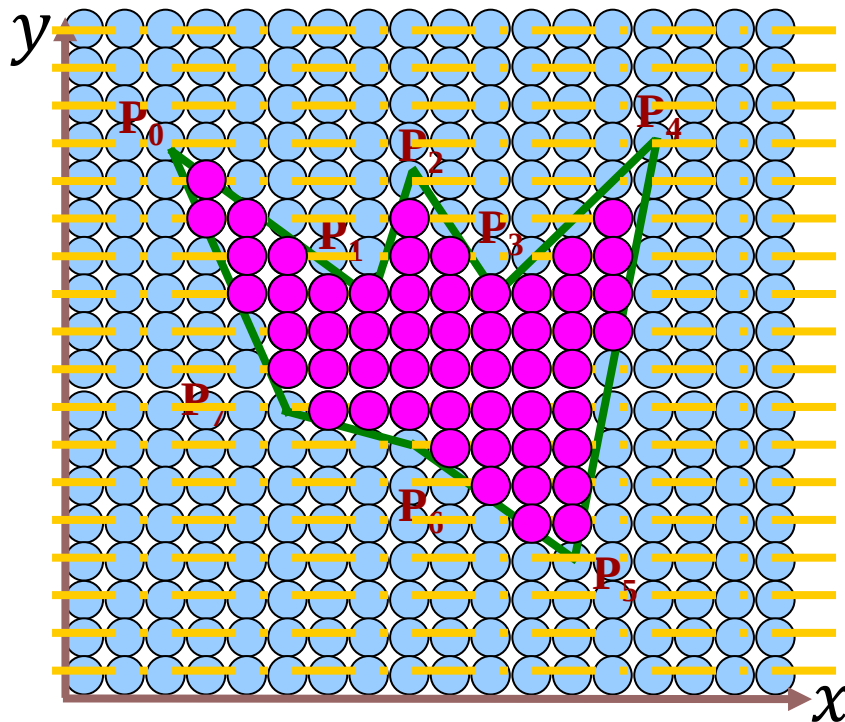
Brute-force scan-line算法

- 求多边形的最小矩形包围盒
- 逐行处理矩形中的像素(扫描线)
 - 求扫描线与多边形边的交点;
 - 对交点从左到右排序
 - 逐对交点配对并绘制交点间的像素
- 连贯性(Coherence)?
 - 上述算法用到了扫描线连贯性
 - 优化Brute-force scan-line算法 → 活化(性)边表扫描线算法

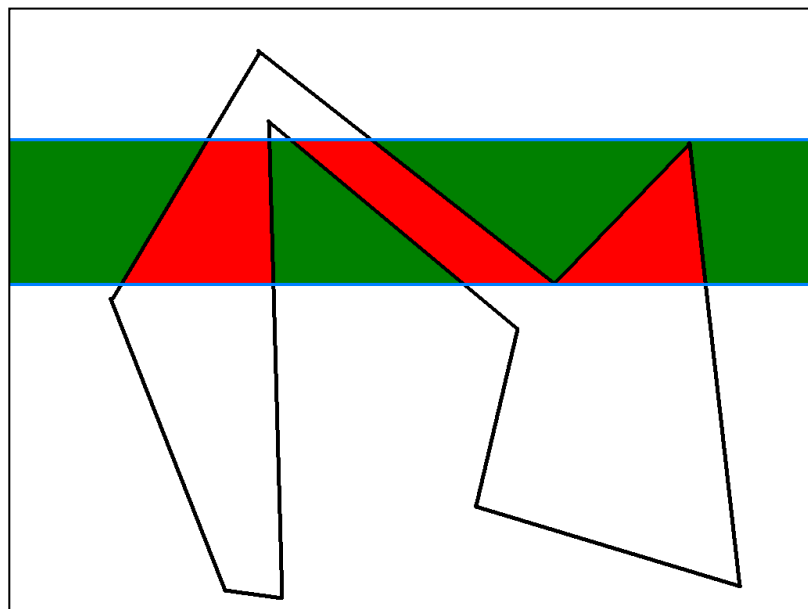


连贯性(coherence)

- 相邻的几何元素往往有共同的性质
 - 避免对像素的逐点判断和求交运算，提高算法效率
- 3个连贯性
 - 区域连贯性
 - 扫描线连贯性
 - 边的连贯性



区域连贯性

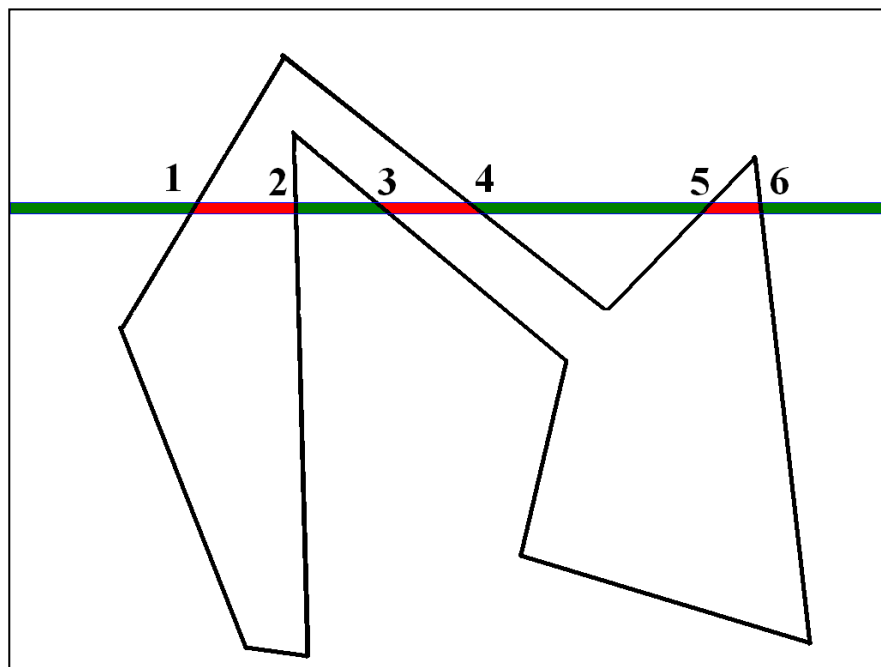


区域的连贯性

- 梯形分为两类
 - 多边形内部
 - 多边形外部
- 两类梯形相间排列
 - 相邻梯形必有一内、一外
- 连贯性运用
 - 按顺序逐条处理扫描线

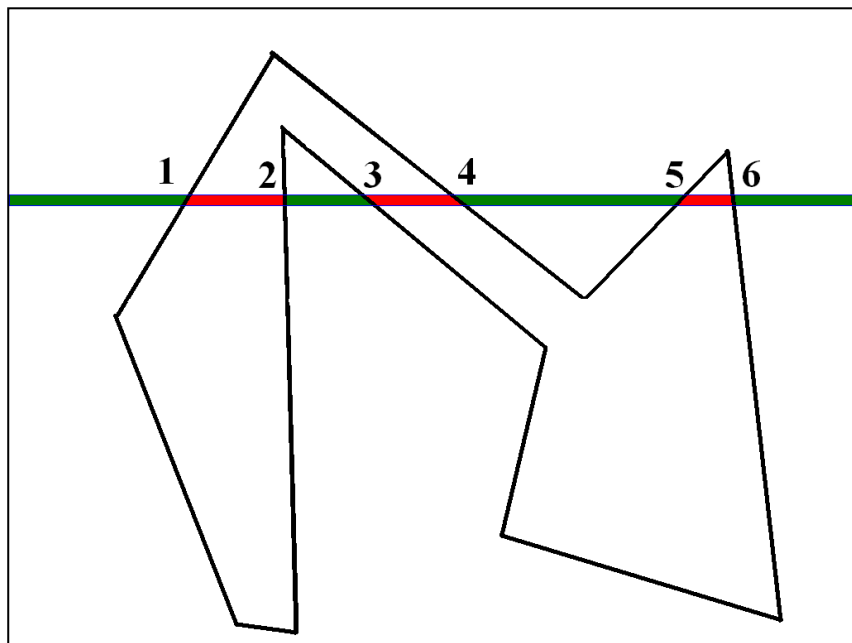
扫描线连贯性

- 区域连贯性在一条扫描线上的反映



扫描线的连贯性

扫描线连贯性



扫描线的连贯性

● 交点序列

- 扫描线与多边形的交点个数为偶数
(1, 2, 3, 4, 5, 6)

● 内外部区间

- 红色区间 (1, 2)、(3, 4)、(5, 6) 位于多边形内部
- 绿色区间位于多边形外
- 两类区间相间排列

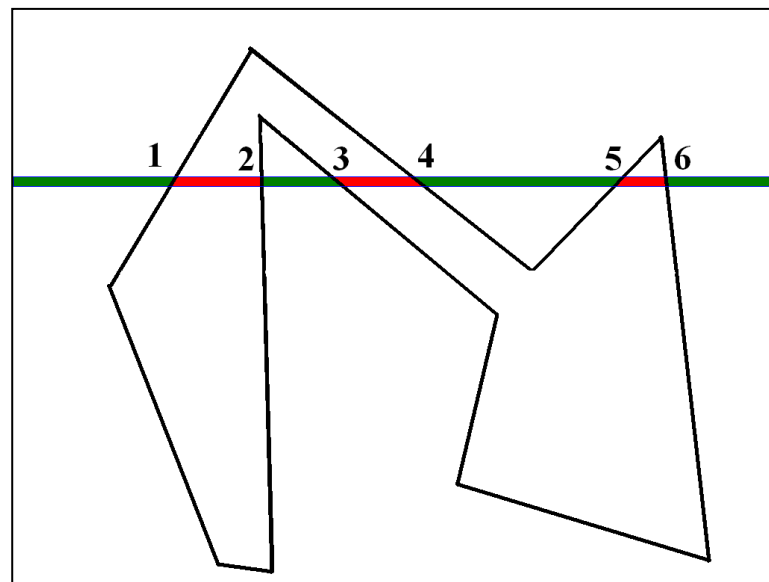
扫描线连贯性

● 推论

- 如果上述交点区间属于多边形内(外), 那么该区间内所有点均属于多边形内(外)。

● 效率提高的根源

- 逐点判断 → 区间判断



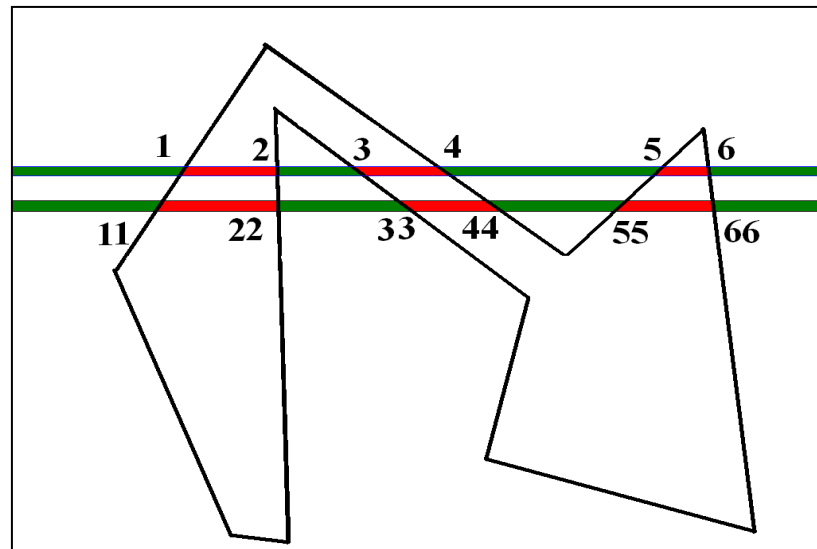
多边形边的连贯性

● 边连贯性

- 直线线性性质在光栅上的表现
- 用于提高多边形与扫描线求交

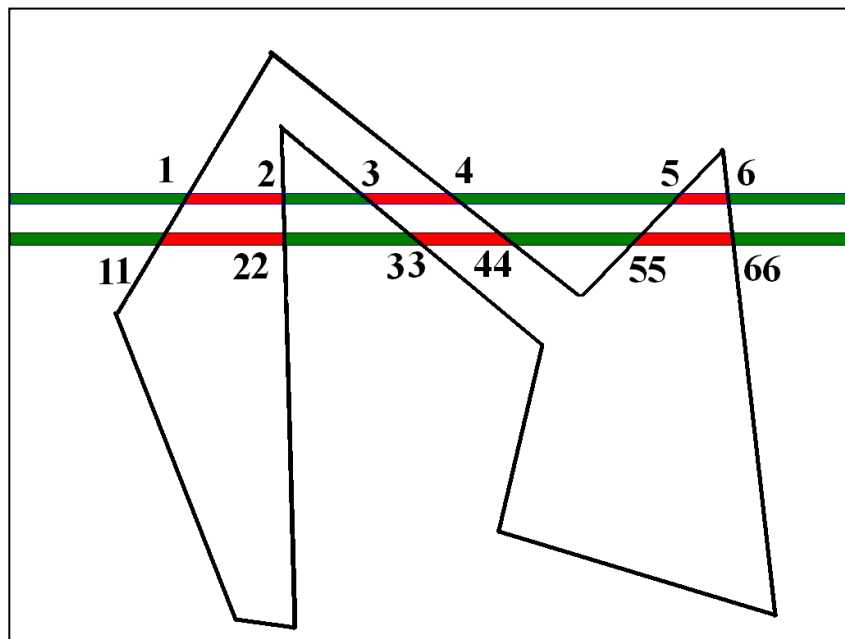
设扫描线 $y = y_{11}$ 与两条边的交点：11 (x_{11}, y_{11});

扫描线 $y = y_1 = y_{11} + 1$ 与同一条边的交点：1 (x_1, y_1);



边的连贯性

边的连贯性



边的连贯性

- 相邻扫描线($y = y_1$; $y_1 = y_{11} + 1$)与同一条边的交点存在如下关系:

$$\frac{y_1 - y_{11}}{x_1 - x_{11}} = k$$



$$x_1 = x_{11} + \frac{1}{k}$$

- 从当前扫描线交点求下一条扫描线交点(增量算法)

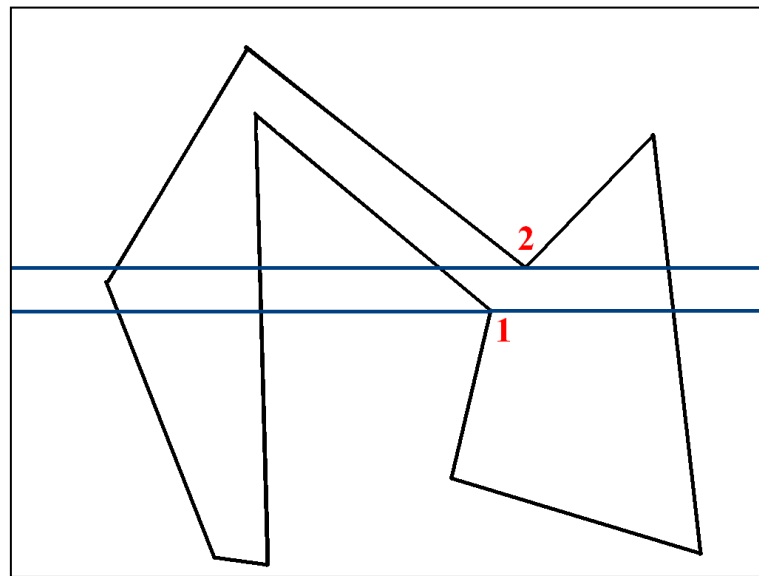
边的连贯性

● 推论

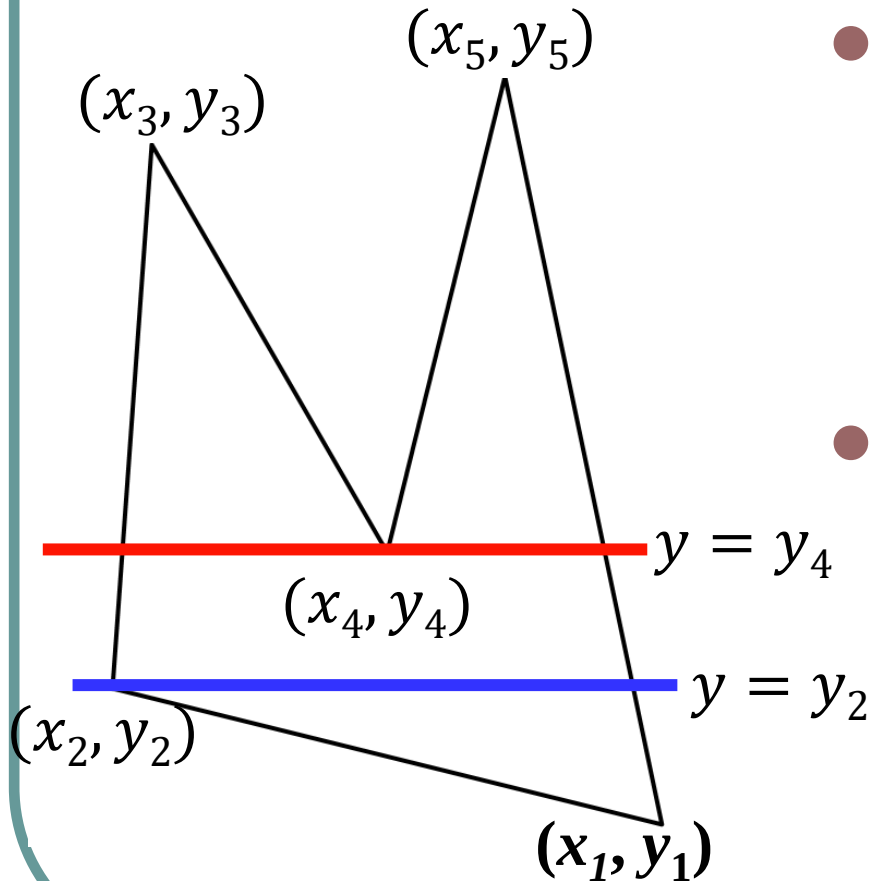
- 边的连贯性是连接区域连贯性和扫描线连贯性的纽带。

● 区域连贯性

- 扫描线连贯性 “+”
边连贯性



多边形顶点分类—极值点

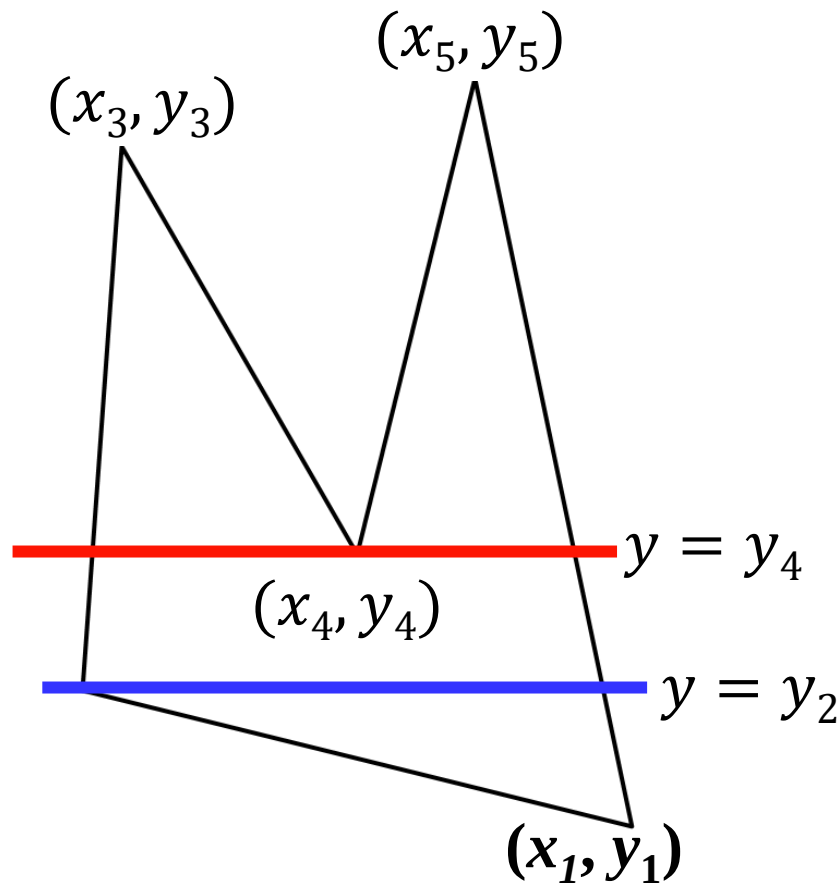


- 顶点 (x_i, y_i) 称为**极值点**
 - 与两个邻点的 y 坐标满足
 $(y_{i-1} - y_i)(y_{i+1} - y_i) > 0$
 即位于扫描线同侧
- 顶点 (x_i, y_i) 称为**非极值点**
 - 相邻三顶点 y 坐标满足：
 $(y_{i-1} - y_i)(y_{i+1} - y_i) < 0$
 即位于扫描线的异侧

奇异点的处理—交点计数规则

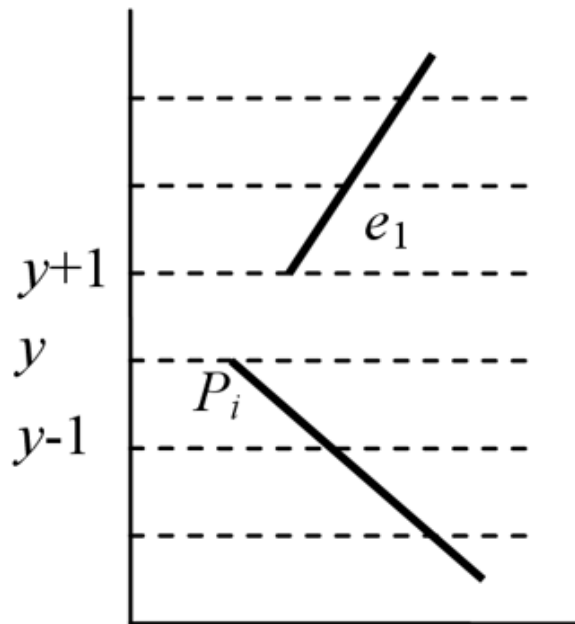
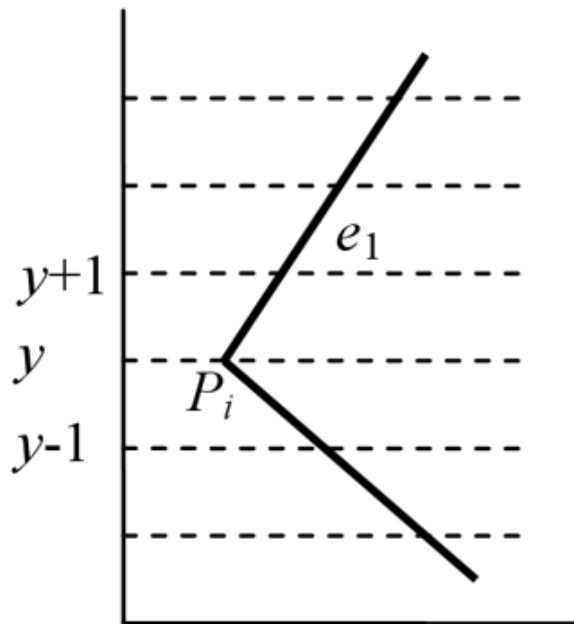
● 奇异点处的交点计数

- 在极值点处，按2个交点计
- 在非极值点处，按1个交点计



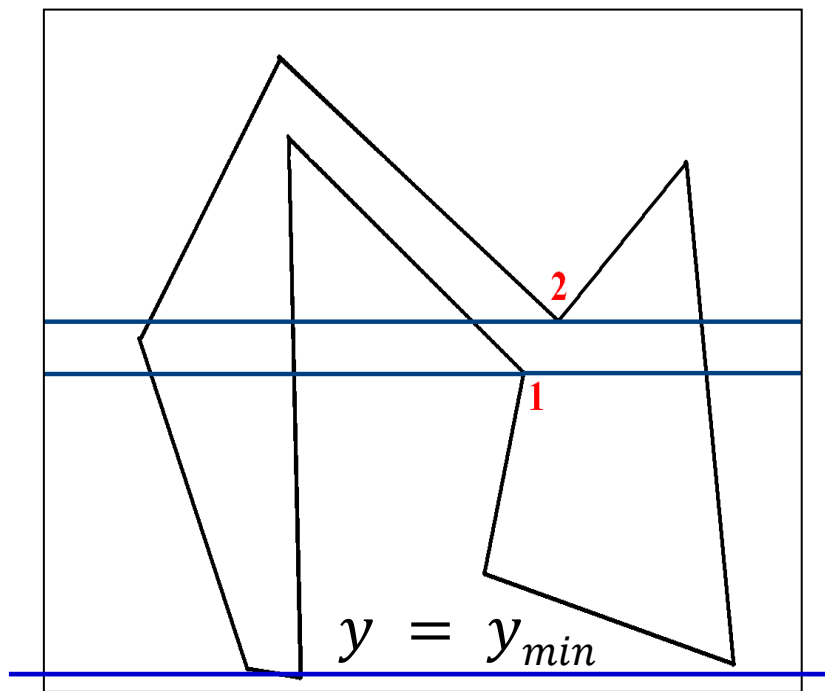
奇异点的处理—(非极值点)预处理

- 将扫描线上方的多边形边在 y 轴方向截断一个单位，使其与当前扫描线无交点



多边形扫描转换算法—基本步骤

- 计算扫描线 $y = y_{min}$ 与多边形的交点
- 根据边的连贯性，按顺序求得各扫描线的交点序列
- 根据区域和扫描线连贯性判断位于多边形内部区段
- 对位于多边形内的直线段进行着色



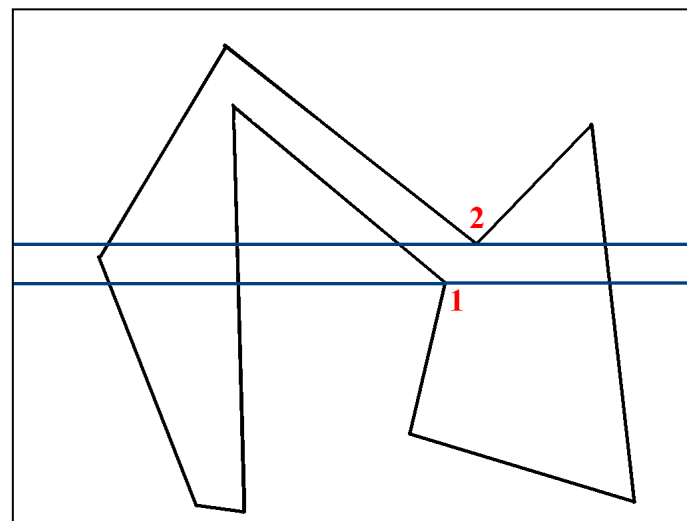
多边形扫描转换算法—数据结构

● 两个表

- 有序边表**ET** (Sorted Edge Table): 记录多边形边的信息, 便于判断是否与扫描线相交, **静态**
- 活化边表**AEL** (Active Edge List): 记录当前扫描线信息, 便于与边求交, **动态**

● 共同基础: 边的结构

```
struct EdgeS{  
    float  $y_{\max}$ ; //边的上端点的 $y$ 坐标  
    float  $x$ ;      //边的下端点 $x$ 坐标  
    float  $dx$ ;     //边的斜率的倒数  
    EdgeS *next;  //指向下一条边  
}
```



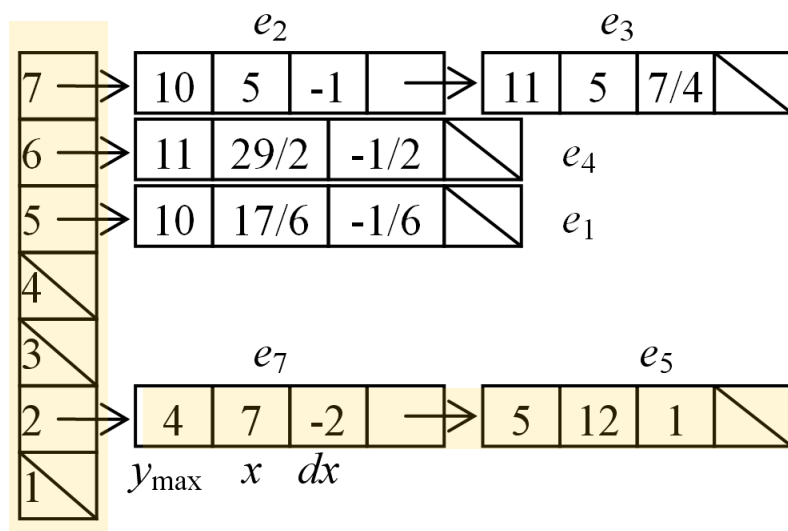
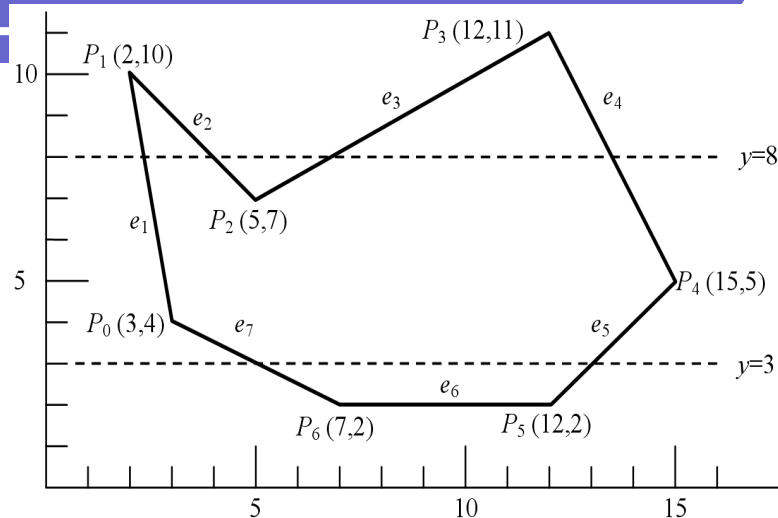
边的分类 (ET)

● 按下端点 y 坐标分类

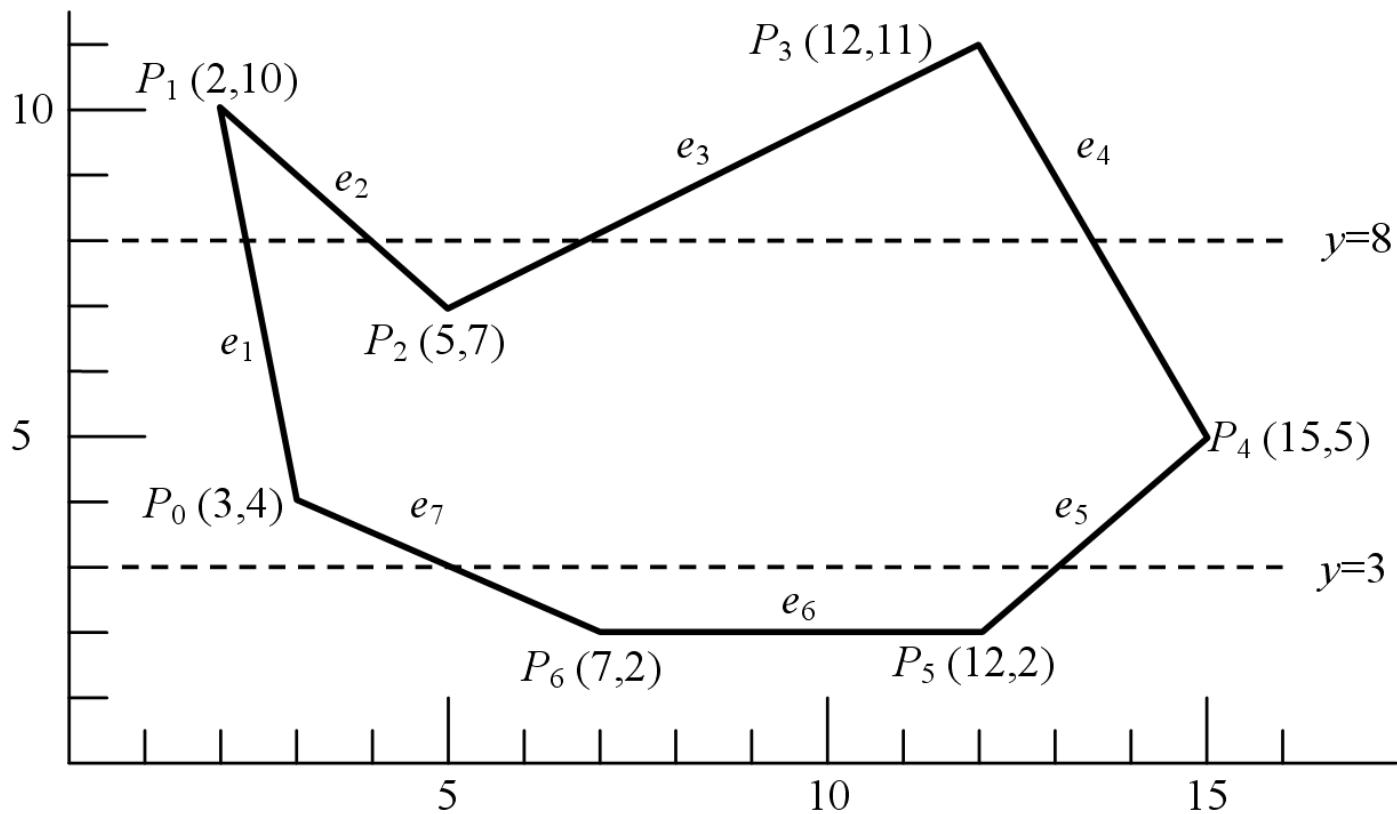
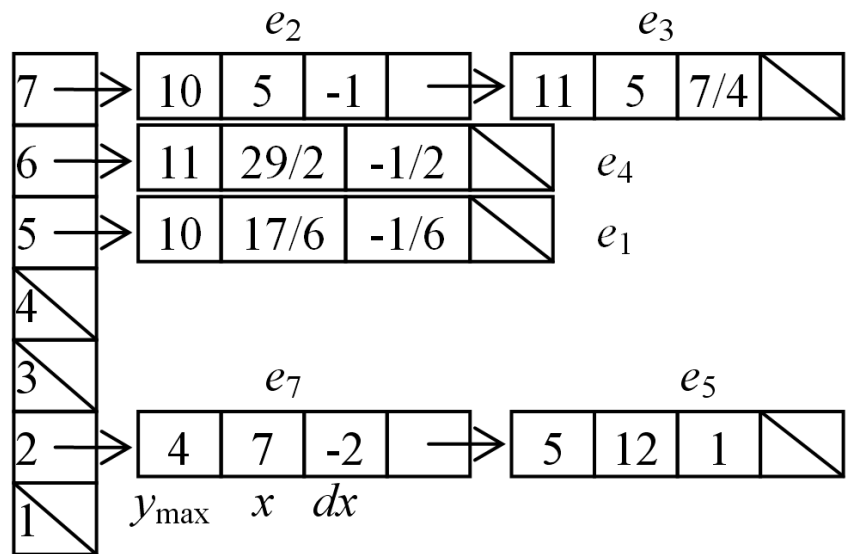
- 该 y 坐标等于 i 的边，归入第 i 类
- 每类构成一个线性链表，用边表指针数组保存
- 水平边不做任何处理

● 同一类边的线性表

- 按 x 值 (x 值相等时按 dx 值) 递增排序

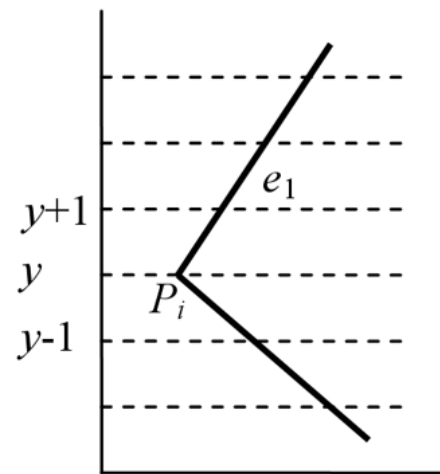


分类的边表实例

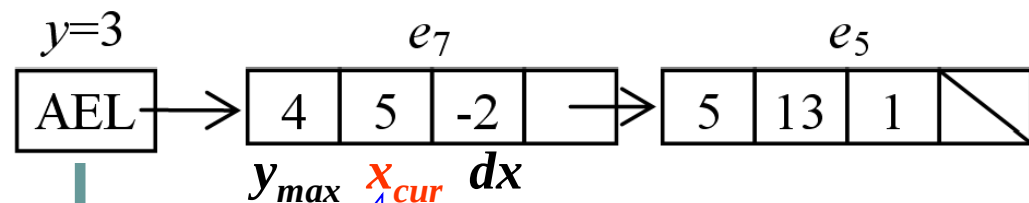
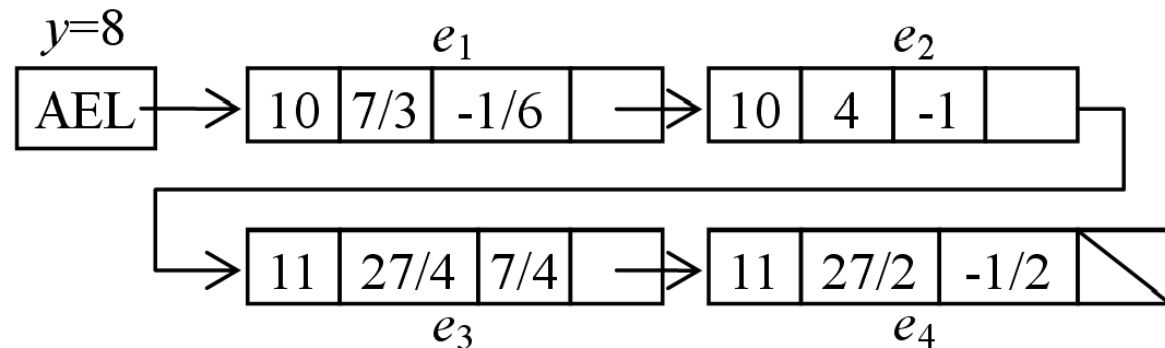


活化边链表(AEL)

- 活化链表由与当前扫描线相交的边组成
 - 记录了多边形的边沿扫描线的交点序列
 - 根据边的连贯性不断刷新交点序列
- 基本单元是(与扫描线相交的)边
- 与分类边表不同
 - 分类边表记录初始状态
 - 活化边表随扫描线移动而动态更新

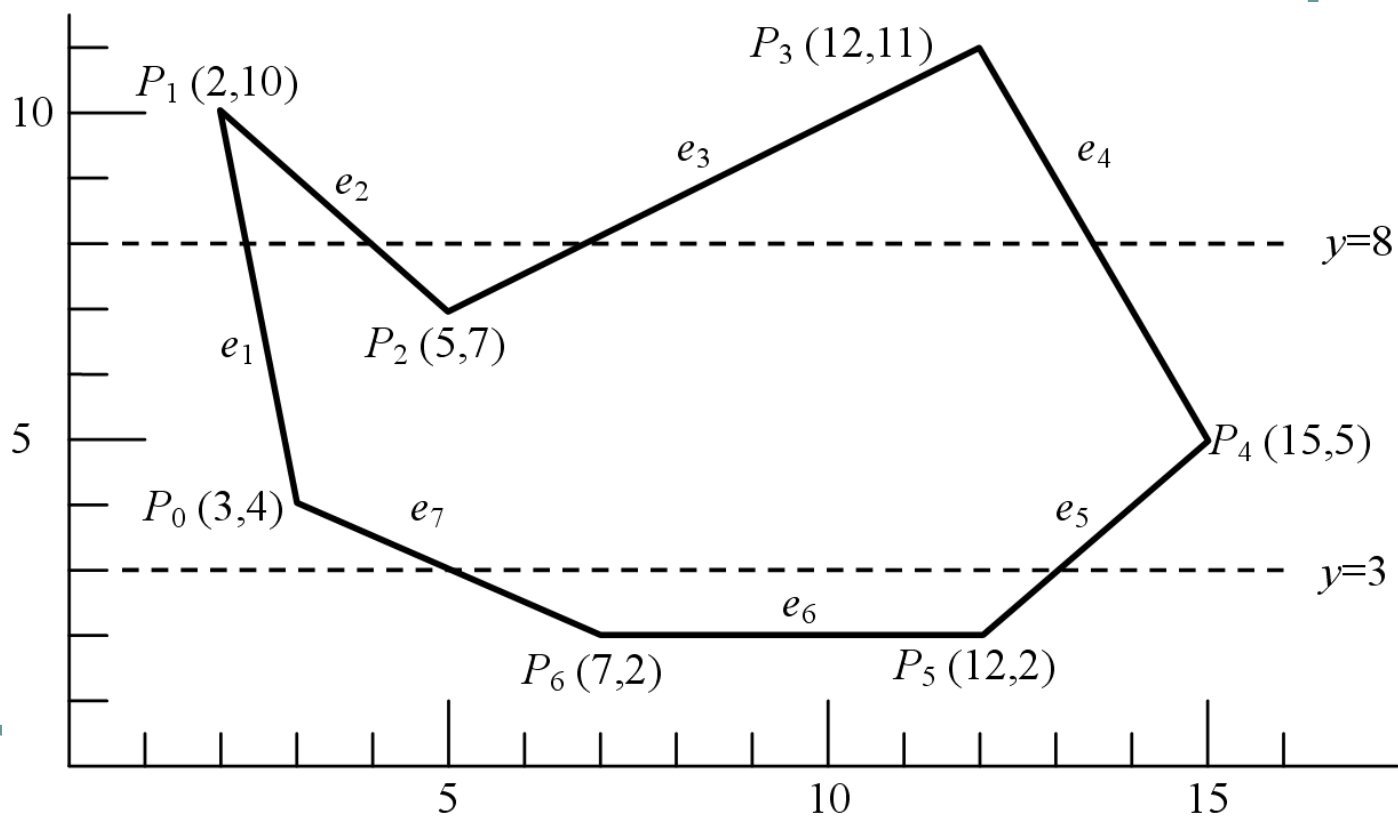


活化边链表实例



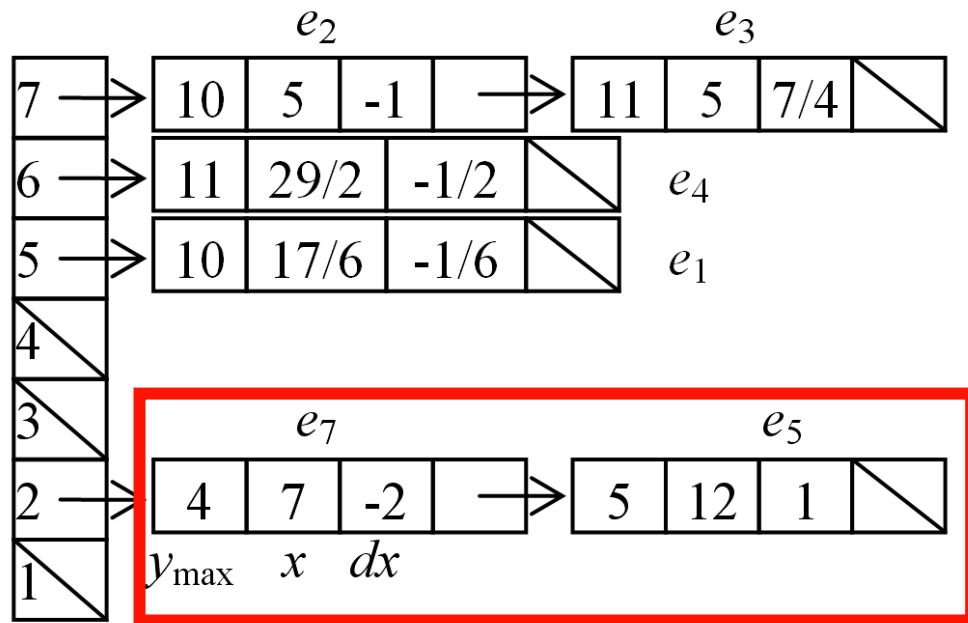
y_{max} x_{cur} dx

与分类边表的区别



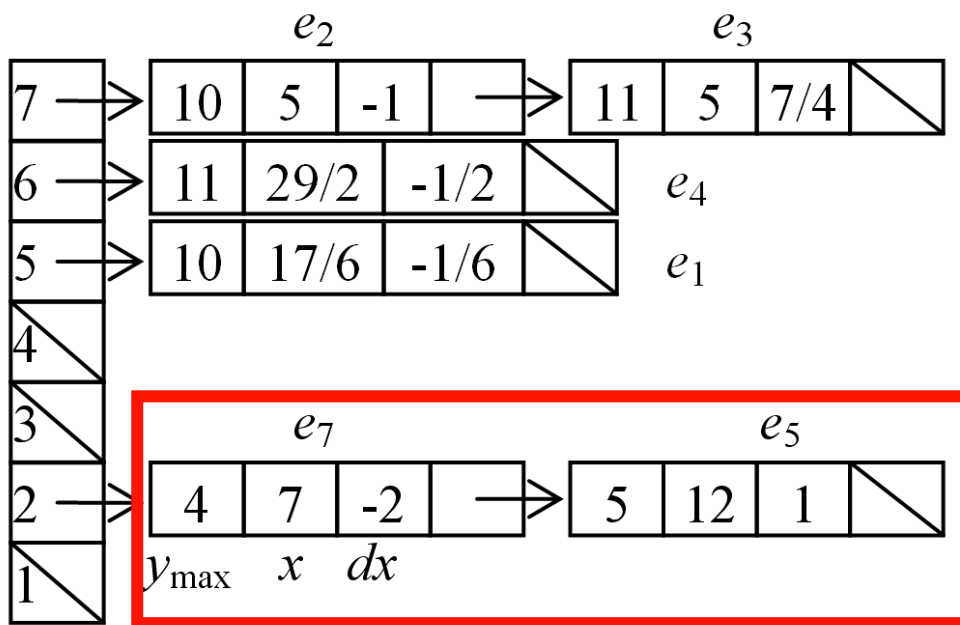
多边形扫描转换算法—细化

- 建立边表(EL);
- 初始化扫描线;
- 初始化活性表(AEL);
- 逐条处理扫描线
 - 取边表到活化表
 - 插入、排序
 - 逐段画内部像素
 - 更新AEL



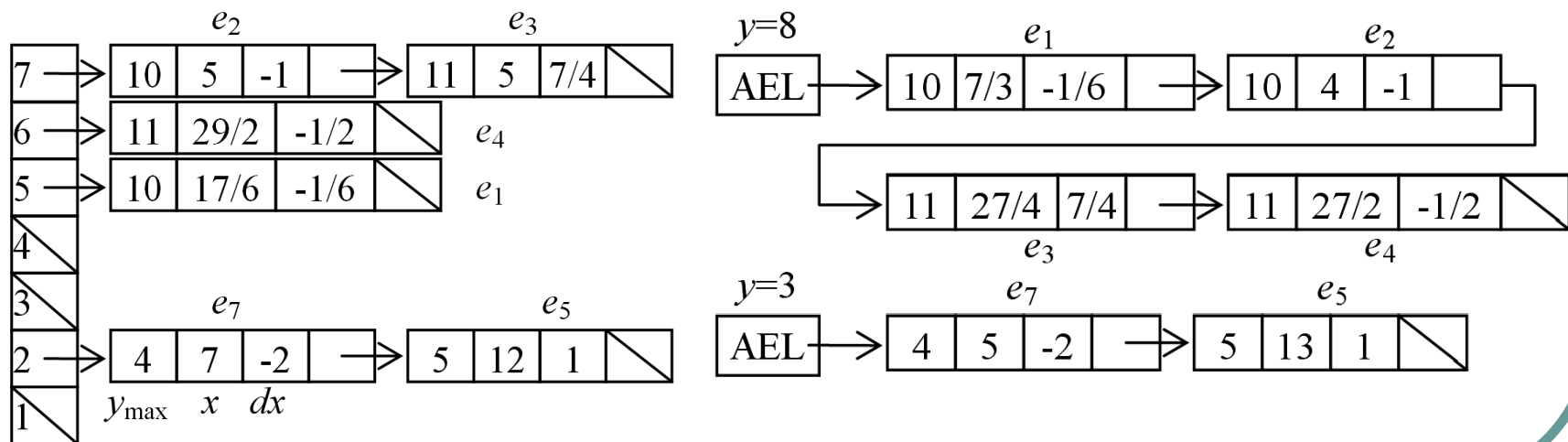
多边形扫描转换算法

1. (y 初始化) 取扫描线纵坐标 y 的初始值为ET中非空元素的最小序号 (本例 $y = 2$)



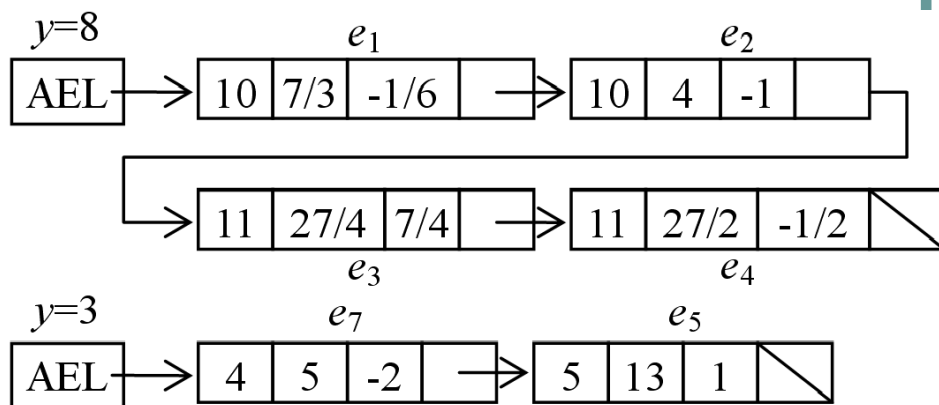
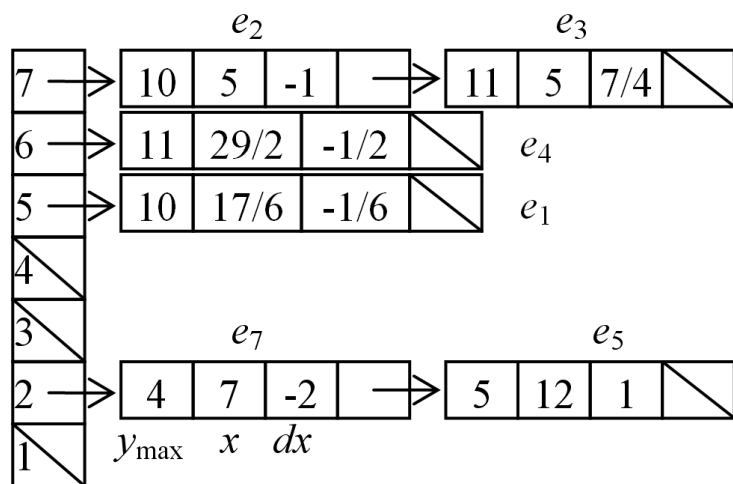
多边形扫描转换算法

2. (AEL初始化) 活化链表AEL置为空
3. 从下到上遍历纵坐标值为 y 的扫描线(当前扫描线)执行如下步骤, 直到ET和AEL为空
 - 3.1 将边表ET中的第 y 类元素取出并插入到活化链表AEL中;
(按 x 值(x 值相等时, 按 dx 值)递增方向排序)



多边形扫描转换算法

- 3.2 将AEL中的边交点两两依次配对,对每对交点间的像素按多边形属性着色;
- 3.3 更新活化链表AEL
 - 3.3.1 删除满足 $y_{max} = y$ 的边;
 - 3.3.2 剩下节点的 x 累加 dx , 即 $x = x + dx$;
- 3.4 将当前扫描线的纵坐标值 y 累加, 即 $y = y + 1$ 。

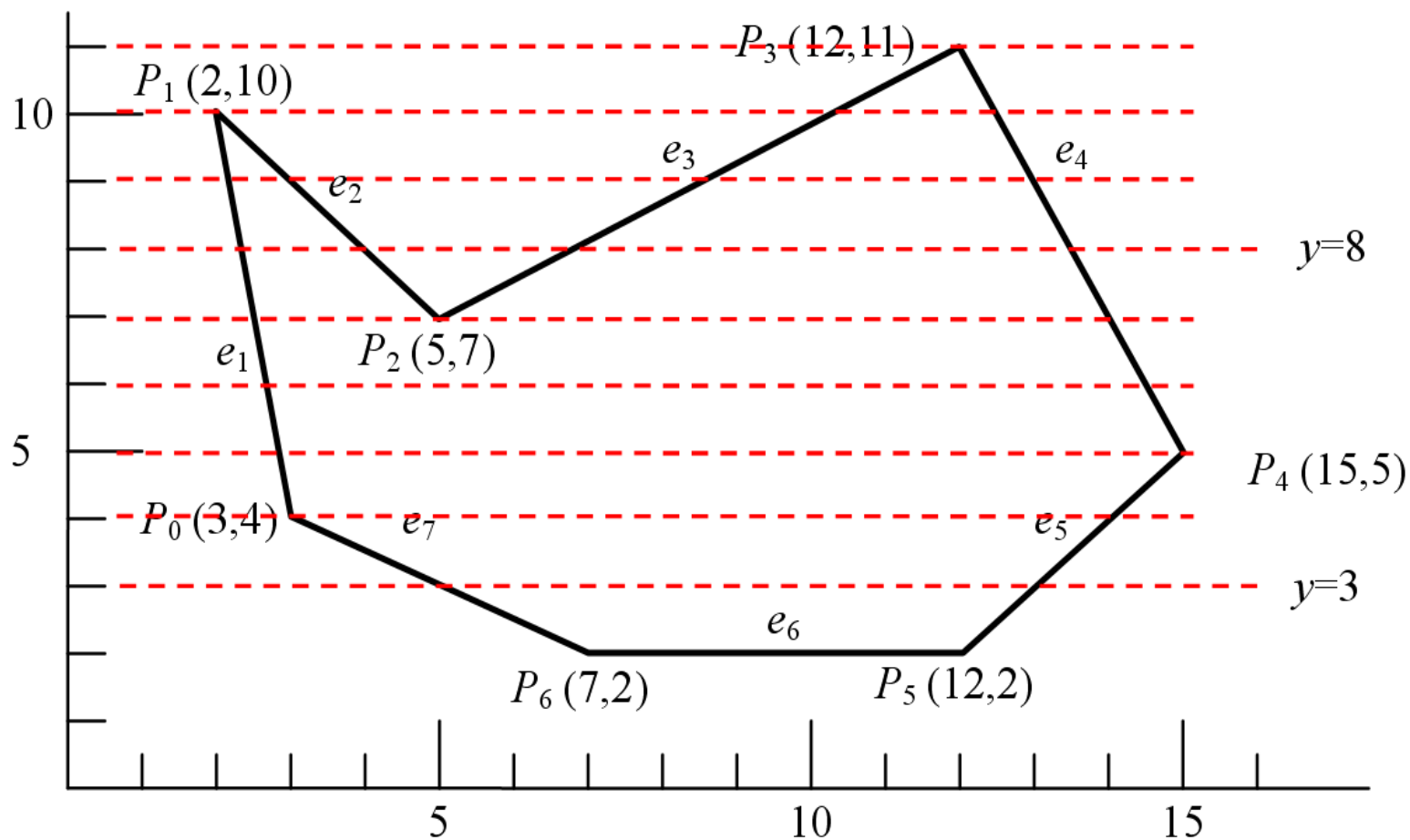


Pseudo code

PolygonScanLineFill(polygon, color)

1. 求最低和最高扫描线号 y_{min} 和 y_{max}
2. 对边按扫描线值作桶分类, 建立边表数组 $ET[y_{min}..y_{max}]$;
3. 初始化AET为空;
4. for $y = y_{min}$ to y_{max}
 5. 修改AET各结点的 x 坐标值;
 6. 把 $ET[y]$ 加入AET;
 7. 对AET按 x 坐标值排序, x 坐标相同则按 dx 排序;
 8. 根据AET逐对填充区间段;
9. endfor

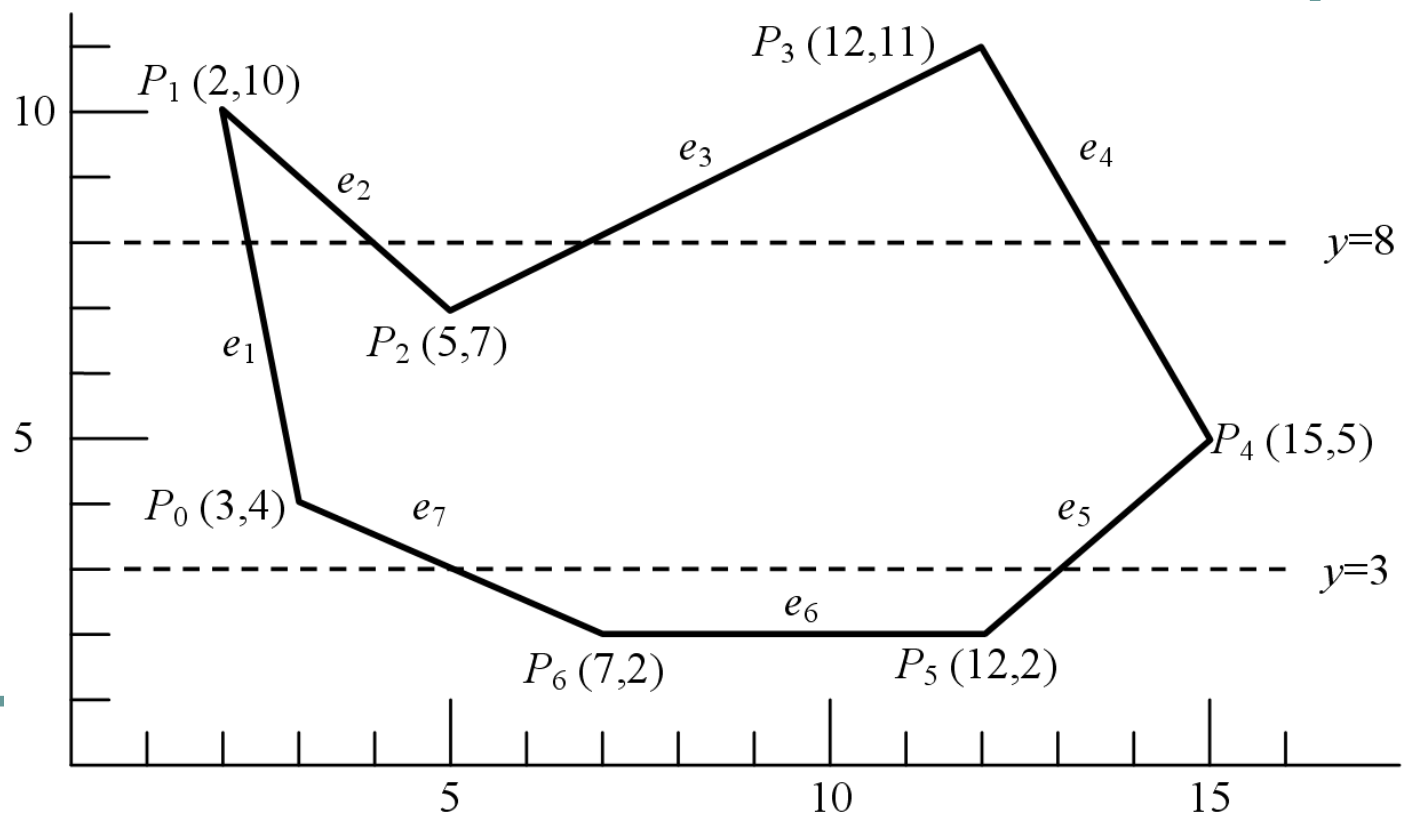
多边形扫描转换实例



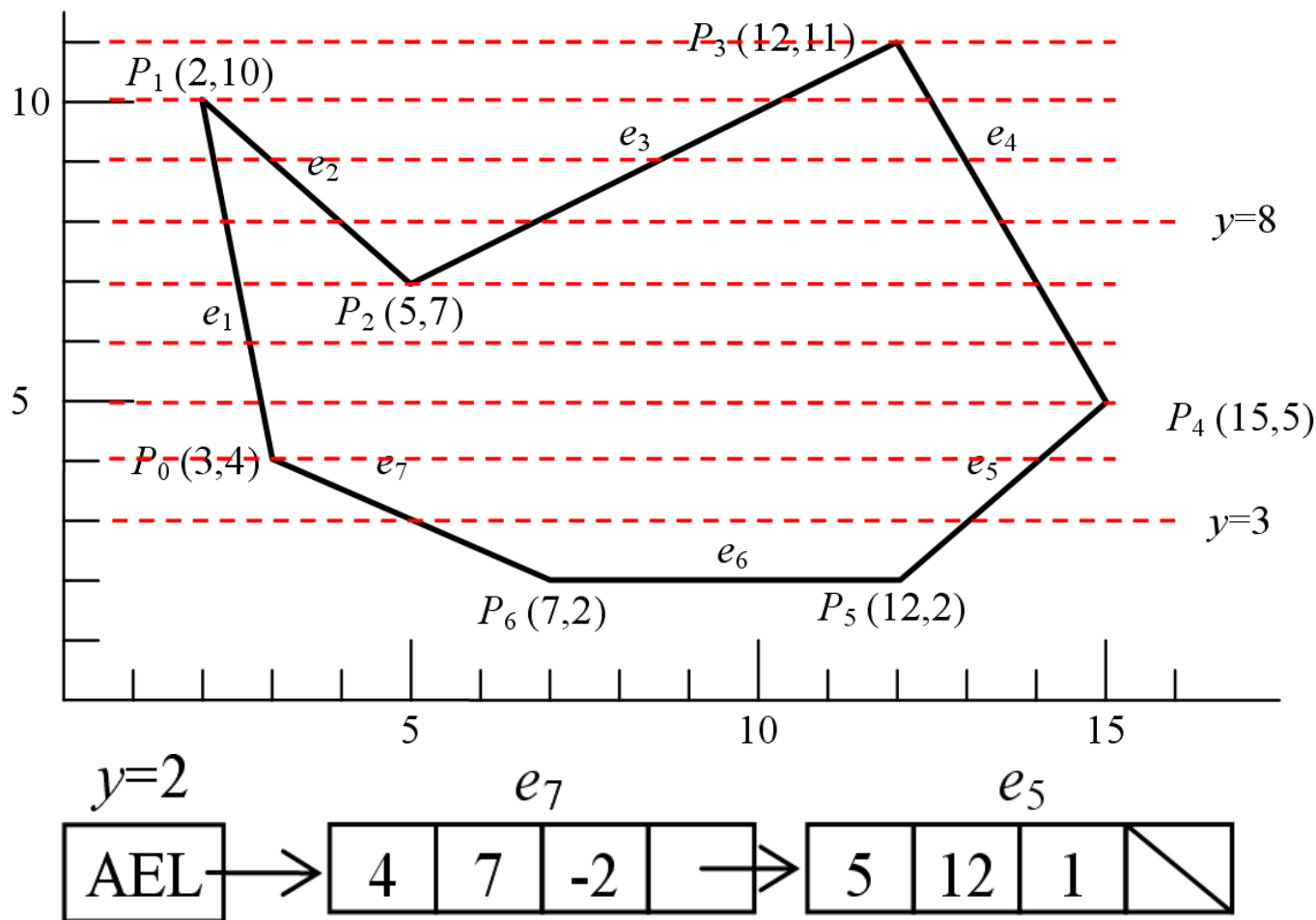
边表

		e_2					e_3			
7	→	10	5	-1	→		11	5	7/4	↘
6	→	11	29/2	-1/2	↘		e_4			
5	→	10	17/6	-1/6	↘		e_1			
4	↘									
3	↘									
		e_7					e_5			
2	→	4	7	-2	→		5	12	1	↘
1	↘									

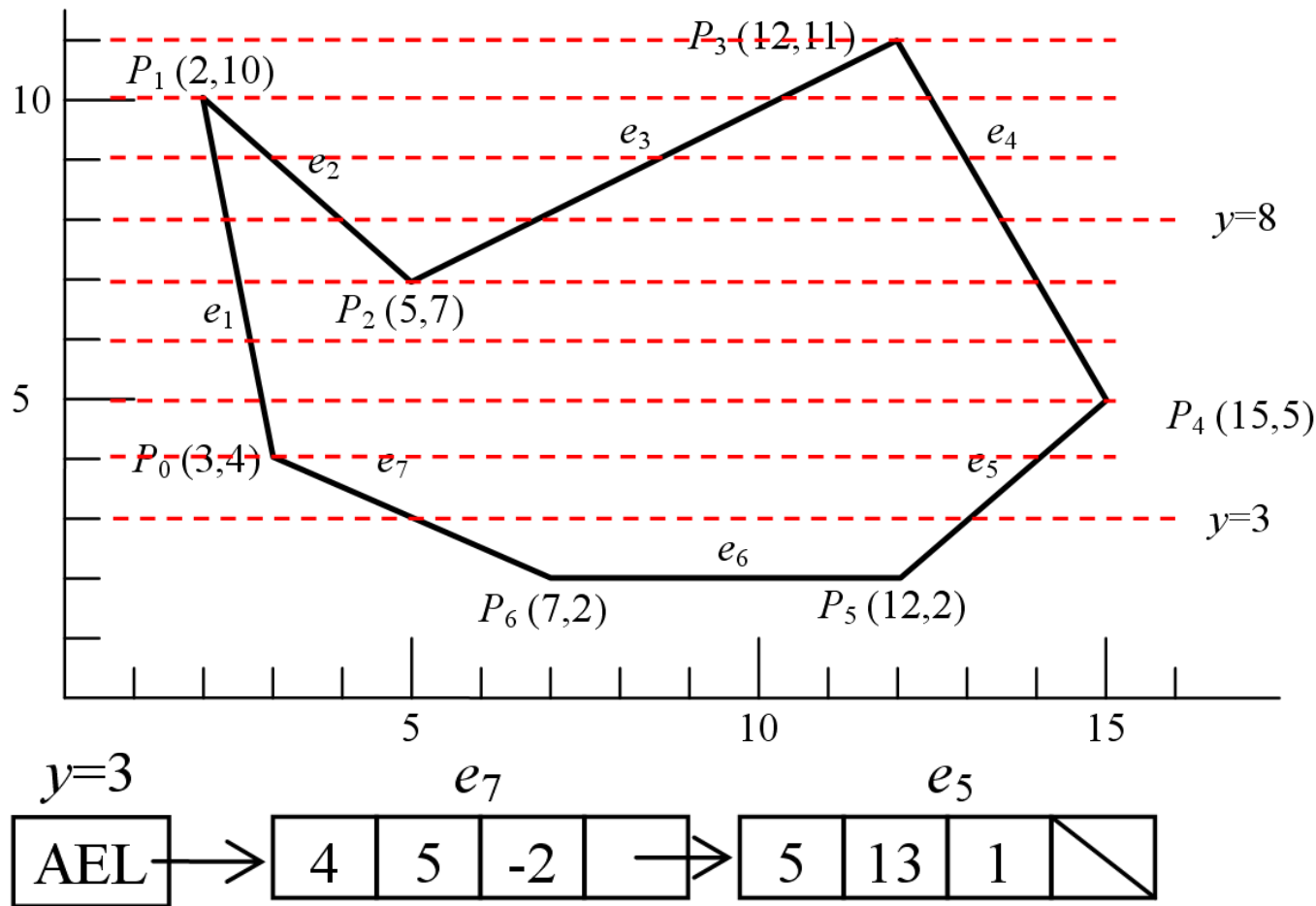
$y_{\max}$ x dx



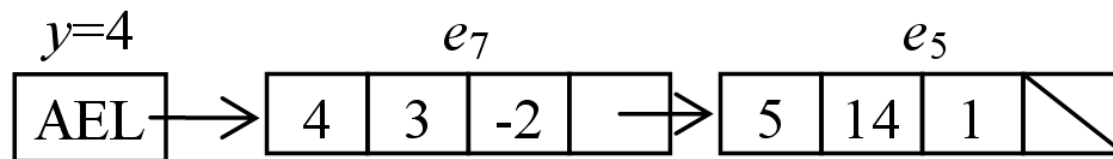
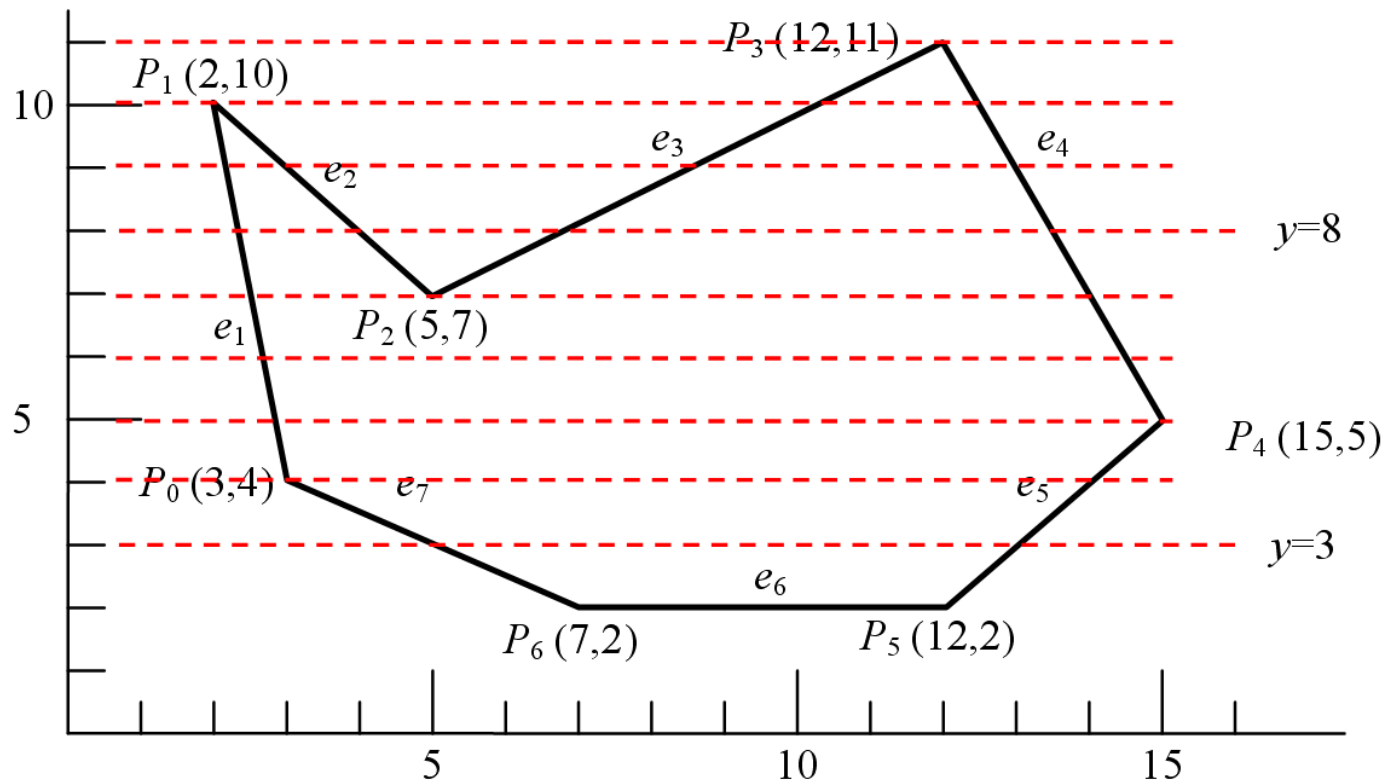
第一条扫描线的活性表



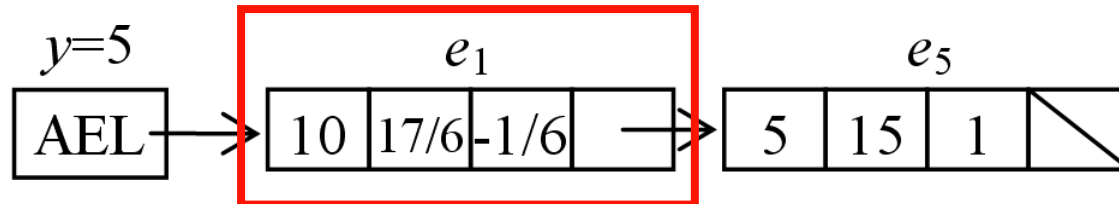
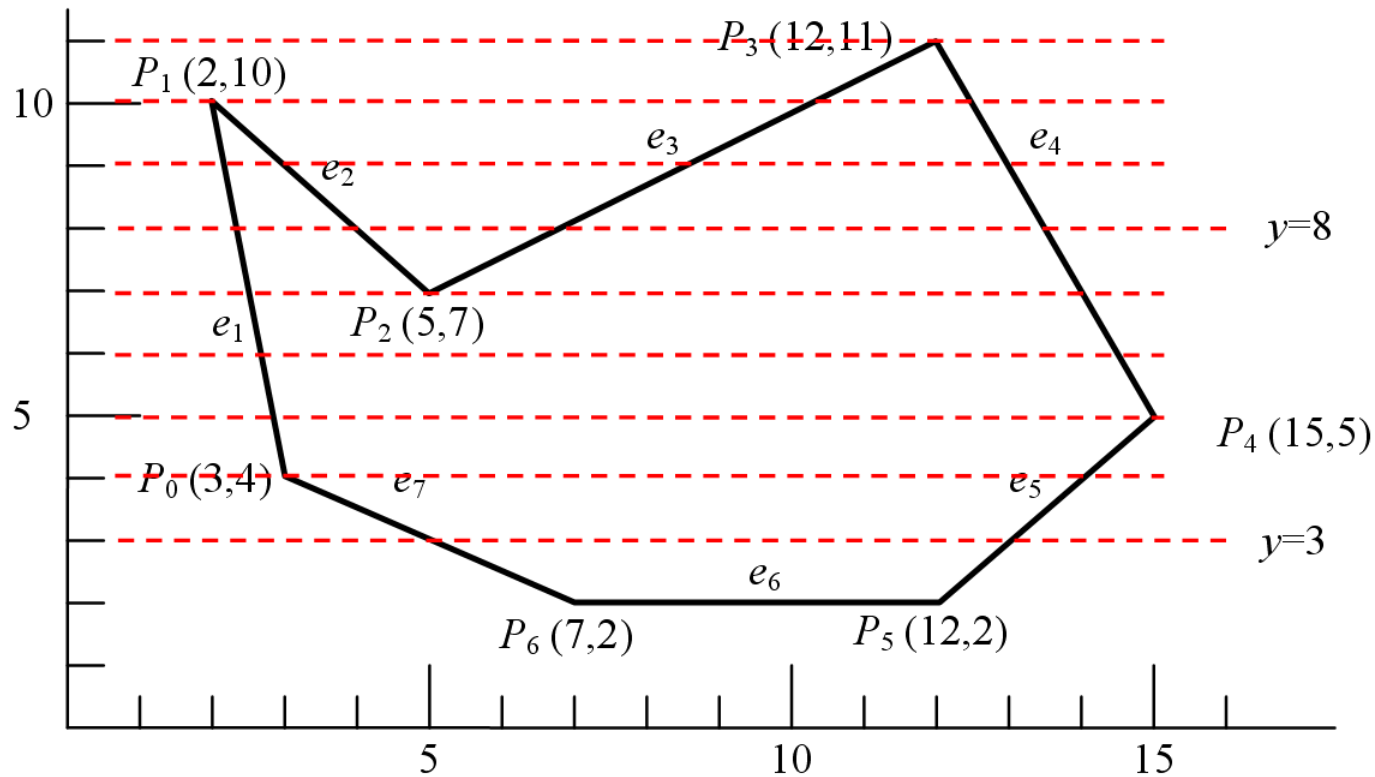
第二条扫描线的活性表



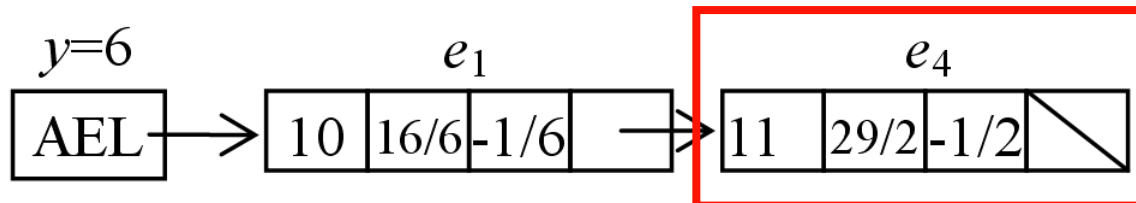
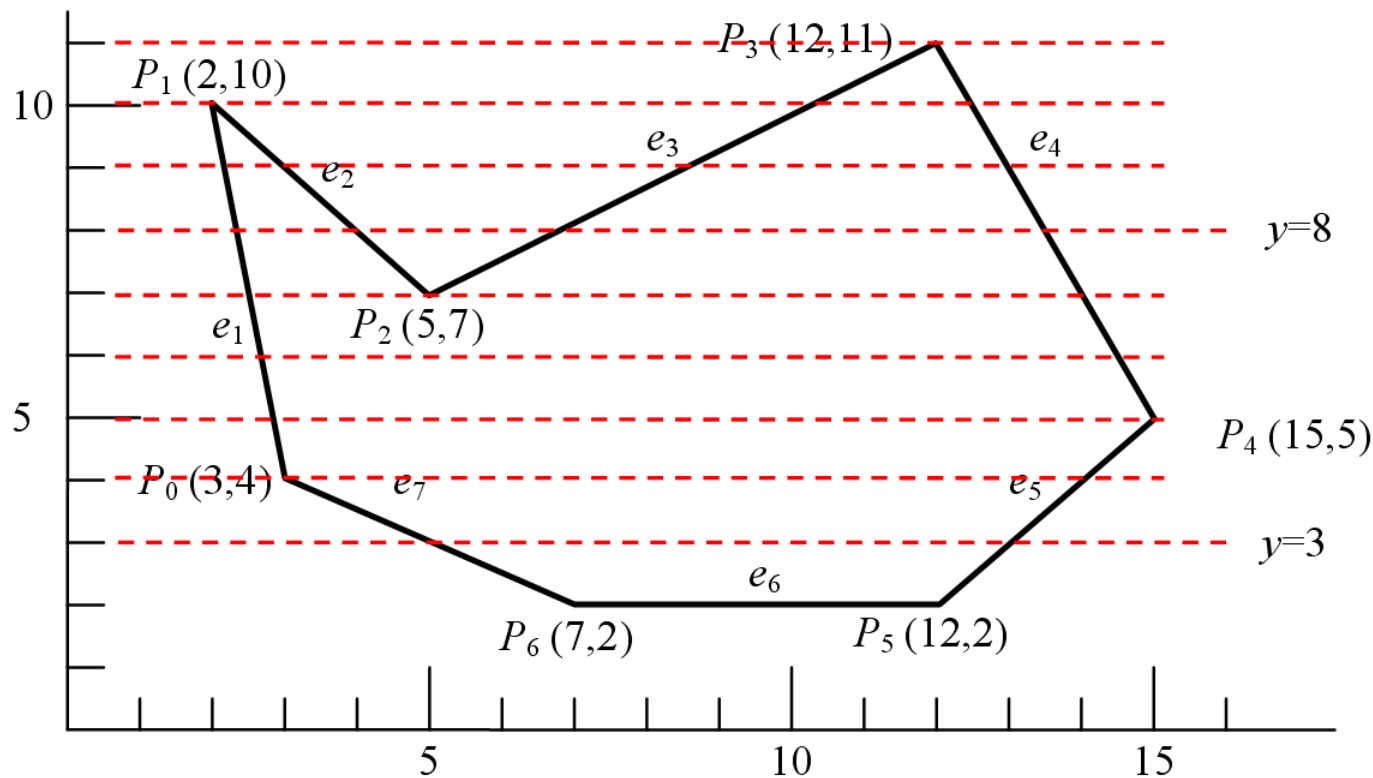
第三条扫描线的活性表



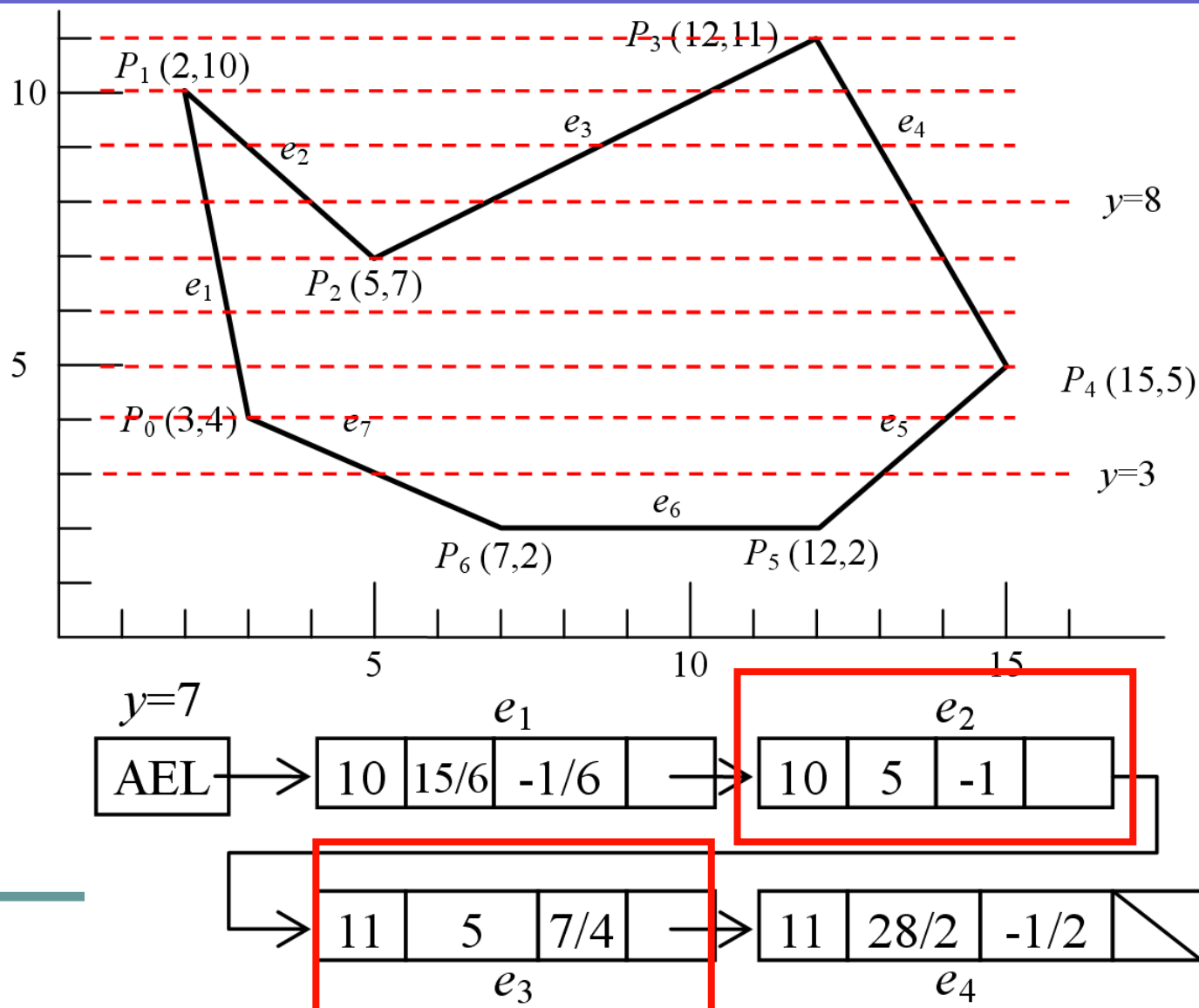
第四条扫描线的活性表



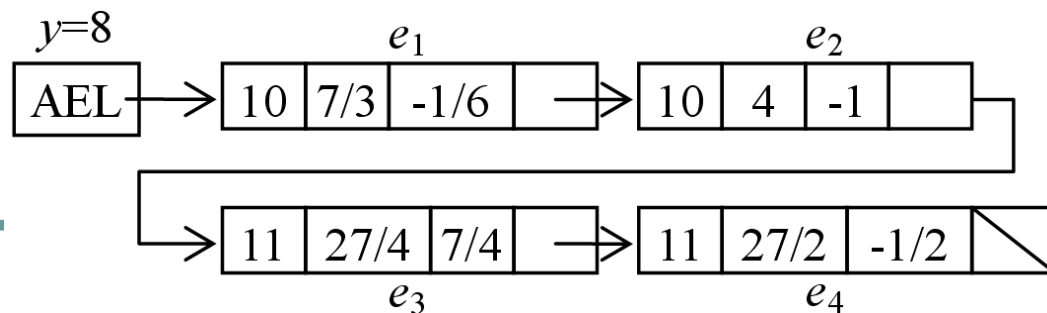
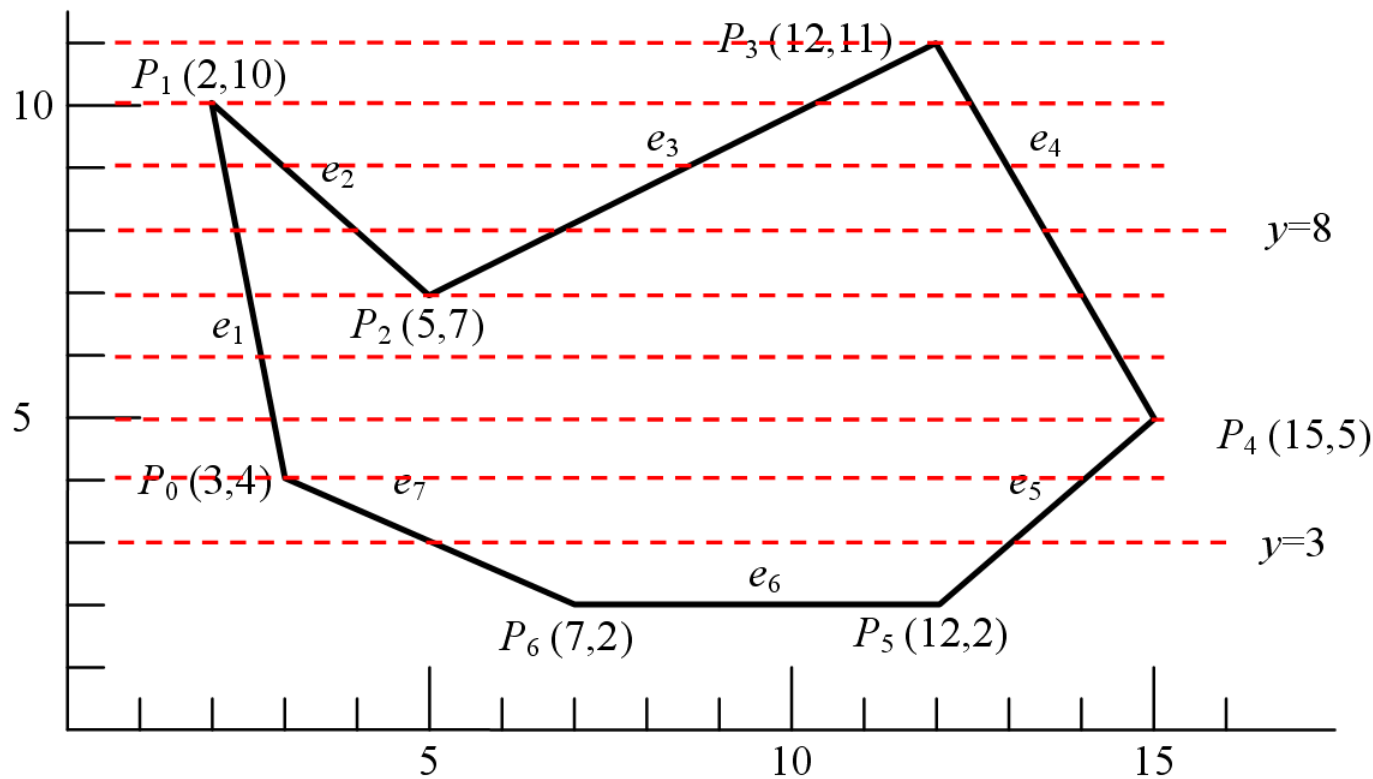
第五条扫描线的活性表



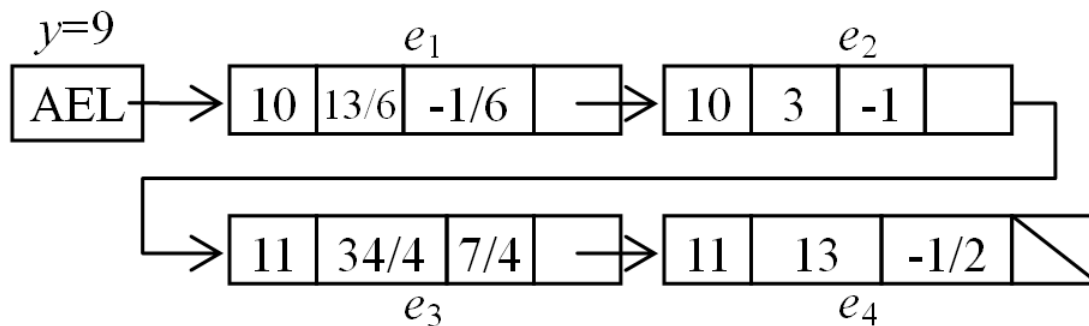
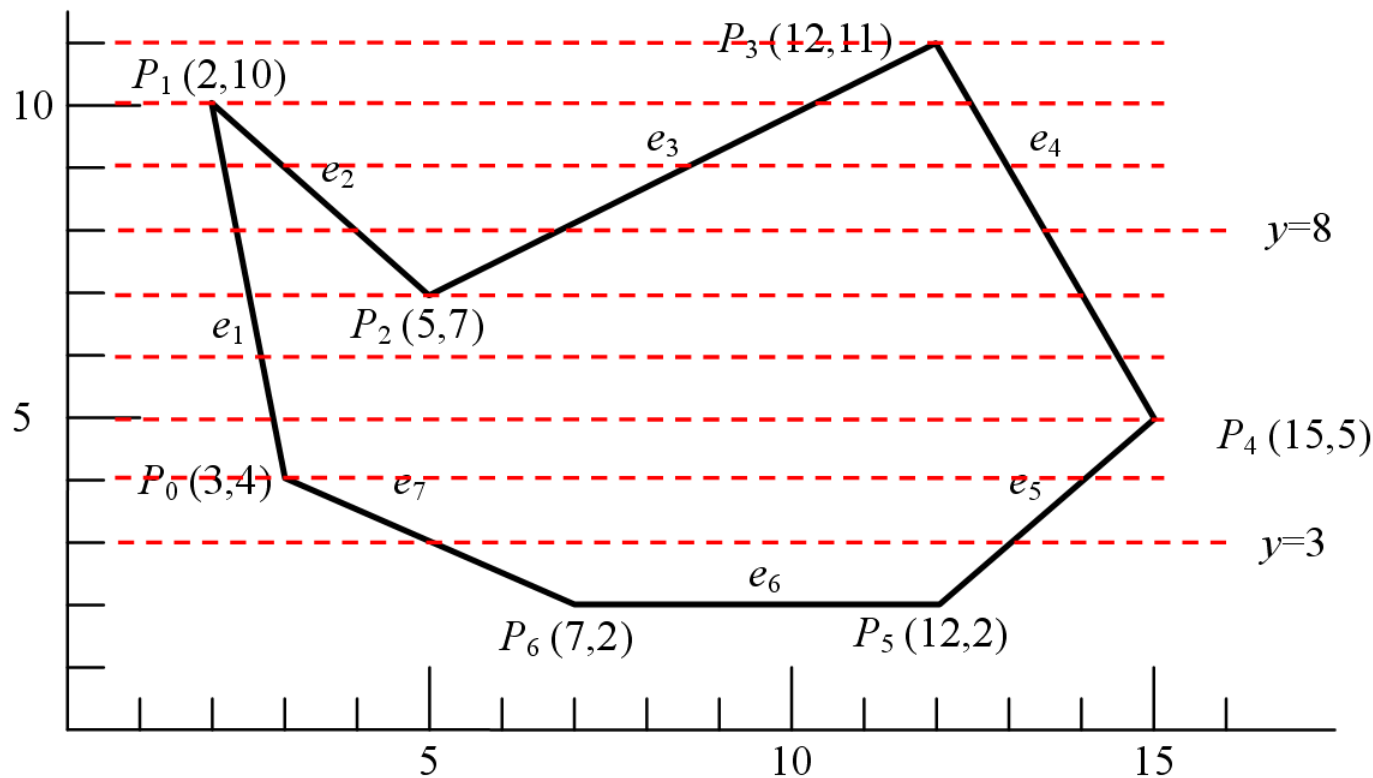
第六条扫描线的活性表



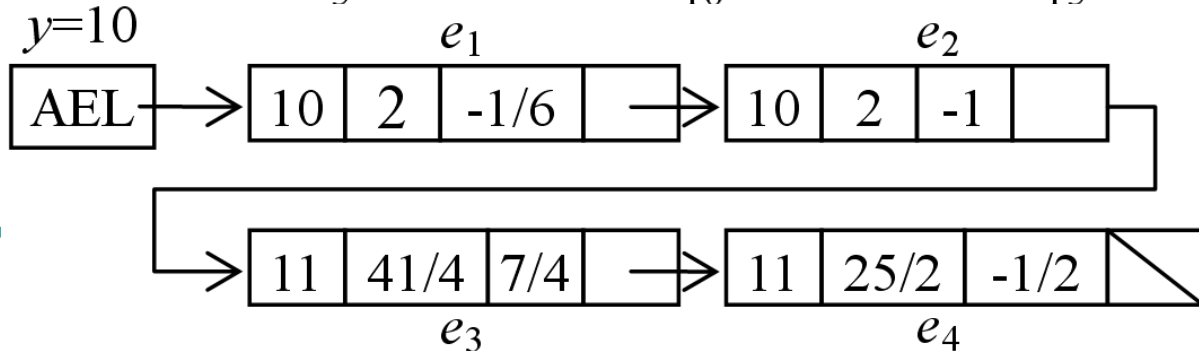
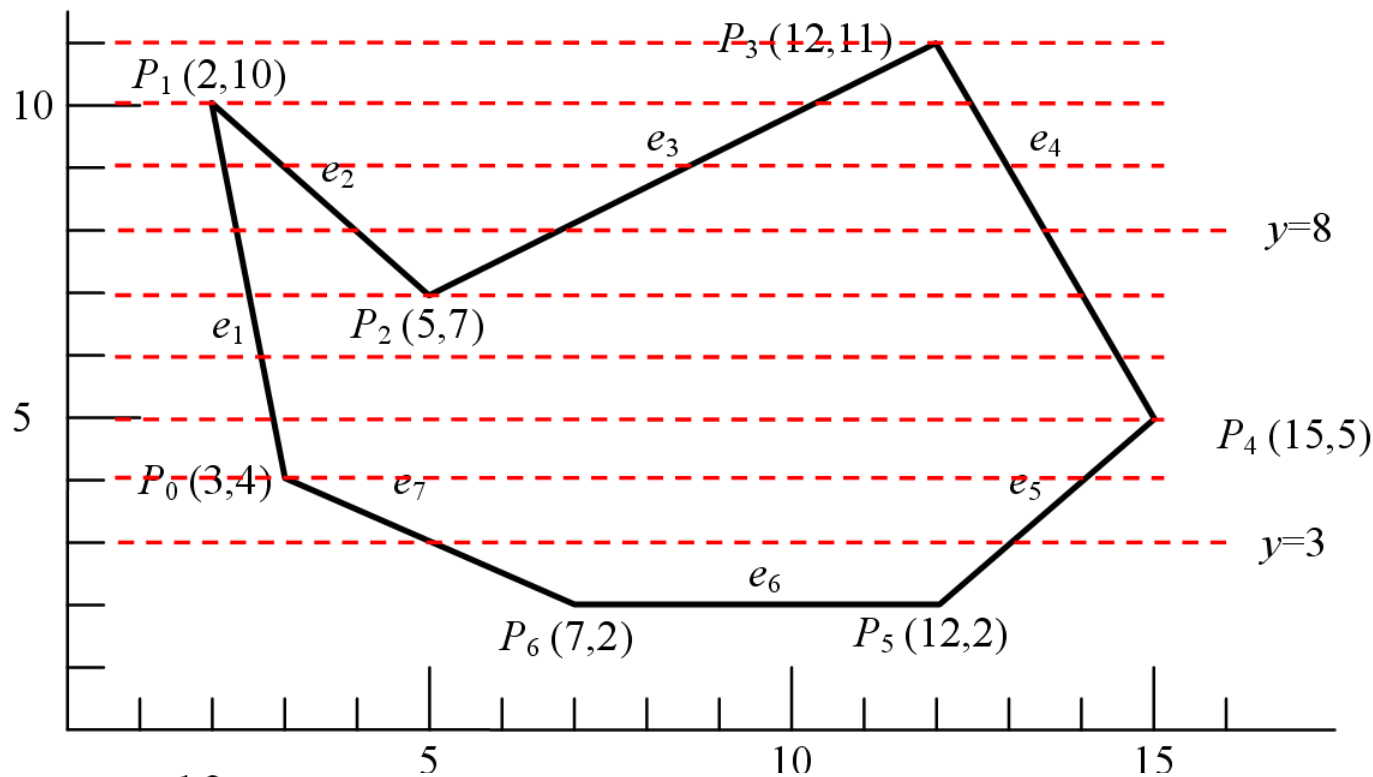
第七条扫描线的活性表



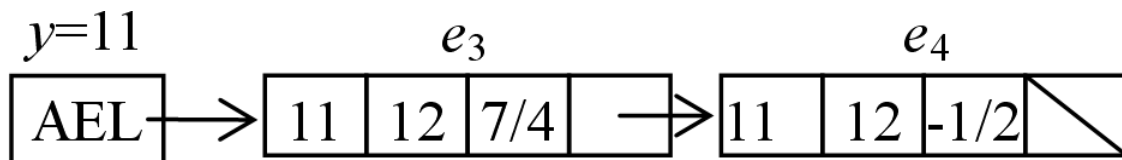
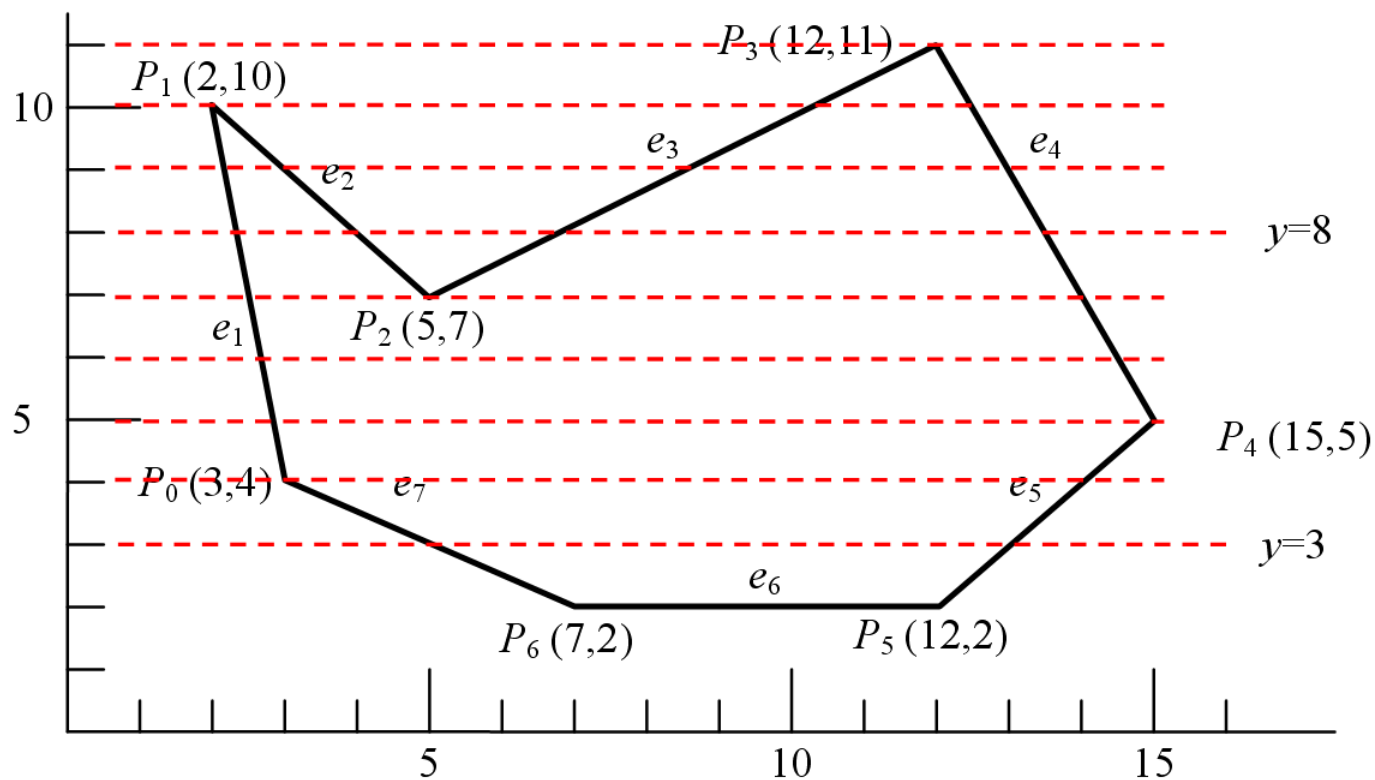
第八条扫描线的活性表



第九条扫描线的活性表



第十条扫描线的活性表



多边形扫描转换的优缺点

● 优点

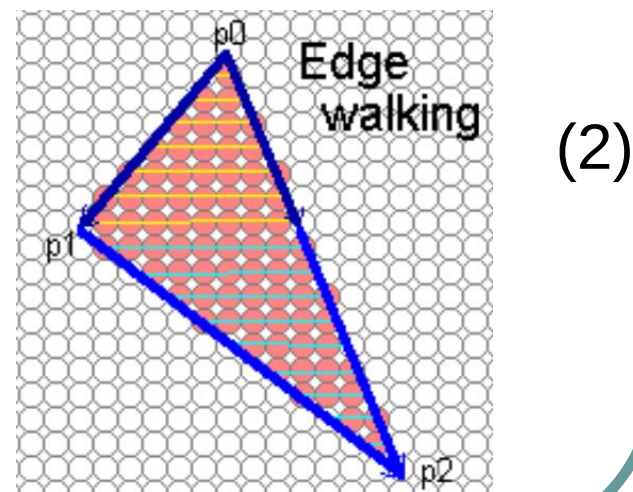
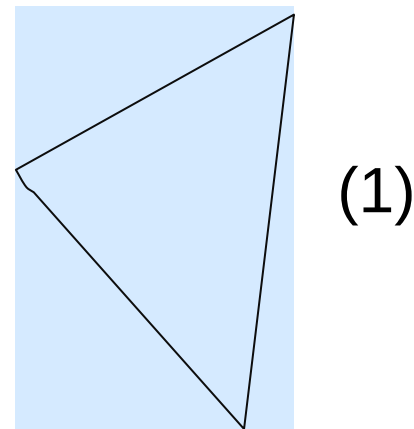
- 充分利用多边形的区域、扫描线和边的连贯性，避免了反复求交的大量运算

● 不足

- 算法的数据结构和程序结构复杂
- 对各种表的维持和排序开销太大，适合软件实现而不适合硬件实现

特殊情形：三角形扫描转换算法

- 三角形网格模型最常用
 - 频繁涉及三角形扫描转换
- 扫描算法
 - 直接用多边形扫描转换算法
 - 内部点判断算法
 - 特殊扫描转化算法(scan conversion)
 - 便于硬件实现



内容

- 基本概念
- 区域填充
- 多边形的扫描转换
- 多边形扫描转换与区域填充的比较

多边形扫描转换与区域填充比较

● 基本思想不同

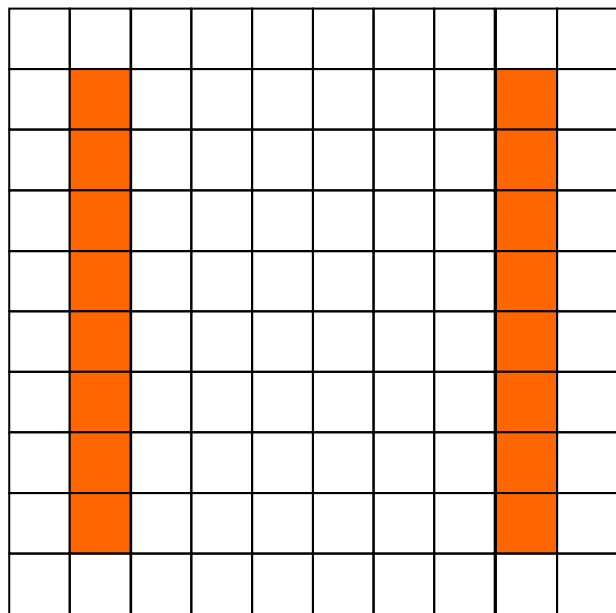
- 多边形扫描转换将多边形顶点表示转换为点阵表示，扫描过程利用了多边形的各种连贯性
- 区域填充只改变区域的颜色，不改变区域的表示方法。填充过程利用了区域的连贯性

多边形扫描转换与区域填充比较

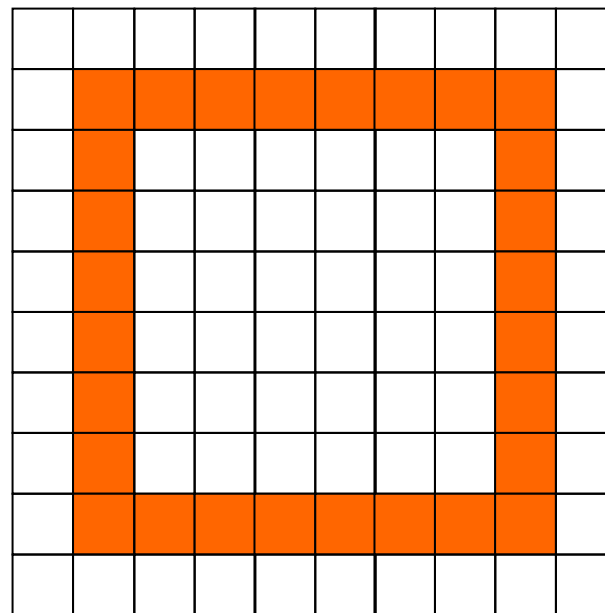
- 对边界的要求不同

- 多边形扫描转换只要求每一条扫描线与多边形有偶数个交点
- 区域填充中
 - 四连通区域必须是封闭的八连通边界
 - 八连通区域必须是封闭的四连通边界

多边形扫描转换与区域填充比较



多边形扫描转换允许边界



区域填充允许边界

多边形扫描转换与区域填充比较

- 出发点不同

- 区域填充：需要区域内一个种子点(复杂计算)
- 多边形扫描转换：没有要求

总结

- 基本概念
- 区域填充
 - 四连通区域和八连通区域
 - 连通区域的种子填充算法
- 多边形的扫描转换
 - 逐点判断算法
 - 扫描线算法
 - 区域、扫描线、边的连贯性
 - 奇异点的处理
 - 数据结构与算法实现
- 多边形的扫描转换与区域填充的比较